

MAS Platform Specification

Authoritative prose rendering from `platform/contracts/platform-spec.yaml`

MARCH 26, 2026

Contents

platform

vocabulary

guard

on_fail_actions

action

timer

required_fields

optional_fields

agent_role

types

flow

required_files

state

required_fields

optional_fields

contract_formats

loader_defaults

file_layout

structure

minimum_flow_package

required_files

optional_files

optional_directories

entity_schema

example

event_schema

format

persistence_model

required_agents

hello_world_example

package_yaml

schema_yaml

nodes_yaml

events_yaml

underscore_prefix_convention

handler_specification

dependency_graph

graph

atomicity

on_complete_vs_rules

short_circuits

handler_fields

description_field

guard

accumulate

compute

on_complete

advances_to

sets_gate

data_accumulation

emits

rules

fan_out

query

reduce

filter

count

clear

action

payload_transform

example

clear_gates

evidence_target

node_specification

node_fields

dependency_rules

core_vs_extensions

core_fields

extension_fields

rules_handler_interaction

engine

flow_tree_walker

algorithm

minimum_required_files

uri_addressing

rules

wildcards

contract_merger

process

agent_sharding

platform_provides

product_provides

dynamic_instance_lifecycle

creation

boot_sequence

steps

event_loop

lifecycle

atomicity

state_management

terminal_state_behavior

accumulation_engine

agent_session_management

session_lifecycle

conversation_modes

timer_model

declaration

lifecycle

example

expression_language

supported_operators

examples

error_model

handler_execution_failure

agent_session_failure

dead_letter_schema

boot_verification

checks

compliance

runtime_enforcement

required_checks

testing

required_tests

handler_execution

rules

product_boundary

rules

flow_coherence

node_coherence

agent_coherence

permissions_model

permissions

bundle_semantics

permission_scoping

messaging

management

routing

tool_model

platform_builtin_tools

custom_tool_schema

prompt_templating

convention

variable_resolution

resolution_order

flow_model

flow_package

nesting

example

pins

required_agents

schema_merging

cross_flow_events

stateless_flows

state_composition

versioning

platform_versioning

product_versioning

compatibility

platform_tables

tables

events

ddl

event_deliveries

ddl

event_receipts

ddl

entity_state

ddl

agents

ddl

agent_sessions

ddl

routing_rules

ddl

mailbox

ddl

spend_ledger

ddl

flow_instances

ddl

timers

ddl

dead_letters

ddl

schema_version

ddl

notes

diagnostics_encoding

runtime_log_encoding

pipeline_transition_encoding

entity_metrics_convention

test_bootstrap

credentials_policy

observation_model

emitted_events

includes

excludes

payload_rules

entity_state

handler_outcome

workspace_model

standard_mounts

/workspace

/data

/opt/mas/contracts

workspace_class_definition

format

rules

mount_guarantees

rules

boot_validation

checks

deployment_mapping

platform

This section defines the platform identity through `name` and `version`.

- `name` : mas-orchestrator
- `version` : 1.3.0

vocabulary

This section defines the platform vocabulary for `state_change`, `guard`, `action`, `timer`, `agent_role`, `flow`, and `state`.

- `state_change` : A transition in the entity state machine, triggered by an `advances_to` handler field.

guard

- `description` : A boolean check evaluated before handler execution. If any guard check fails, the handler is blocked. Guards use CEL expressions evaluated against entity, payload, and policy context.

ON_FAIL_ACTIONS

- `reject` (default)
- `discard`
- `kill`
- `escalate:{event}`

action

- `description` : A platform-provided operation executed as the final step of a handler. Only two valid actions exist: `create_flow_instance` and `record_evidence`. Product-specific actions are not supported.

timer

- `description` : A time-based trigger attached to a stage. When the delay elapses, the platform emits the timer's event, which can trigger a transition.

REQUIRED_FIELDS

- `id`
- `state`
- `event`
- `owner`

OPTIONAL_FIELDS

- `delay_seconds`
- `delay_minutes`
- `delay_hours`
- `delay_days`
- `cancellation`
- `recurring`

agent_role

- `description` : An entity that owns transitions, handles events, and executes logic. Three execution types with different semantics.

TYPES

- `system_node` :
 - `description` : Deterministic code. No LLM. Subscribes to events, executes fixed logic, emits events. Owns workflow transitions. Examples: scoring-node, validation-orchestrator.
 - `execution` : deterministic
 - `has_prompt` : false
 - `has_model` : false
- `agent` :
 - `description` : LLM-powered. Has a system prompt, model tier, conversation mode. Subscribes to events, reasons about them, calls tools. Can own workflow transitions where the stage change requires LLM judgment (e.g. promote/kill decisions, build readiness). Agents produce events that may also trigger transitions owned by system nodes.
 - `execution` : llm
 - `has_prompt` : true
 - `has_model` : true

- `runtime` :
 - `description` : The platform itself. Handles transitions that are implicit (human decisions, system lifecycle). Not declared in any contract — always available.
 - `execution` : implicit
 - `has_prompt` : false
 - `has_model` : false

flow

- `description` : A self-contained building block with input/output pins (events and data), system nodes that orchestrate state transitions, and required agent roles. The marketplace unit. Analogous to an IC chip in circuit design.

REQUIRED_FILES

- `nodes.yaml`
- `events.yaml`
- `schema.yaml`

state

- `description` : A named state in a workflow. An entity (e.g. a vertical, a ticket, a document) is always in exactly one state. States belong to phases for grouping. Some states are terminal (no outgoing transitions).

REQUIRED_FIELDS

- `id`
- `phase`

OPTIONAL_FIELDS

- `description`

contract_formats

Canonical format for all flow contract files.

loader_defaults

- **description** : Fields that are optional in contract YAML because the loader derives them from context.
- **schema_name** : Optional. If omitted, derived from the directory name of the flow package. A flow at flows/scoring/ gets name “scoring” automatically.
- **schema_namespace** : Optional. If omitted, derived from the flow path relative to root. A flow at flows/scoring/ gets namespace “scoring”. Root flow gets empty namespace.
- **agent_id** : Optional. If omitted, the YAML map key IS the agent ID. agents.yaml uses map keys as canonical identifiers: {“opco-ceo”: {model_tier: sonnet, ...}} means agent_id = “opco-ceo”. Explicit id field overrides the map key if present, but this is discouraged.
- **agent_emit_events** : Optional. If omitted, defaults to empty list []. An agent that only observes (subscribes but never emits) may omit emit_events entirely.

file_layout

- **description** : Every flow has the same directory structure. The root flow is the entry point. Flows nest via flows/ subdirectories. The runtime/ bridge provides merged flat files for flat file loaders.

structure

```
{root_flow}/
  package.yaml      # Root flow (entry point)
  schema.yaml       # Manifest: name, version, flows, handoffs, entity_schema
  nodes.yaml        # Root flow pins
  events.yaml       # Root-level system nodes
  agents.yaml       # Root-level events
  tools.yaml        # Root-level agents
  policy.yaml       # Shared tools
  prompts/         # Shared policy
  runtime/         # Shared policy
    nodes.yaml      # Root-level agent prompts
    events.yaml     # Merged flat files for flat file loader
```

```

agents.yaml
tools.yaml
policy.yaml
flows/
  {child_flow}/          # Child flow (same structure, recursive)
    package.yaml
    schema.yaml
    nodes.yaml
    events.yaml
    agents.yaml
    tools.yaml          # Optional
    policy.yaml         # Optional
    prompts/
    flows/              # Grandchild flows (if any)
      {grandchild}/
        package.yaml
        ...

platform/               # Platform contracts (separate from any flow)
  package.yaml
  platform-spec.yaml

```

minimum_flow_package

REQUIRED_FILES

- package.yaml
- schema.yaml

OPTIONAL_FILES

- nodes.yaml
- events.yaml
- agents.yaml
- tools.yaml
- policy.yaml

OPTIONAL_DIRECTORIES

- prompts/
- flows/

- **rule** : A flow with no agents needs no prompts/. A flow with no handlers needs no nodes.yaml. A flow with no events needs no events.yaml.

entity_schema

- **description** : entity_schema in package.yaml declares the persistent fields for entities managed by this flow. The platform derives database DDL from this schema at boot.
- **format** :
 - **groups** : Schema is organized into named groups (e.g., discovery_phase, scoring_phase).
 - **fields** : Each field is declared as: field_name: type_description
 - **types** : text, integer, numeric(precision,scale), boolean, jsonb, timestamp, uuid
 - **annotations** : Parenthetical notes after the type are human-readable, not machine-parsed.

example

```
entity_schema:
  identity:
    vertical_id: uuid (primary key)
    name: text
  scoring_phase:
    composite_score: numeric(5,2)
    discovery_mode: text
```

event_schema

- **description** : Each workflow defines event payload schemas in events.yaml. Events are keyed by event name. Each entry contains a payload object mapping field names to types (string, integer, object, array, boolean). The platform uses these schemas to validate emit tool calls at runtime. Routing, consumers, and delivery are NOT declared in events.yaml — the platform derives routing from agents.yaml (subscriptions, emit_events) and nodes.yaml (subscribes_to, event_handlers).

FORMAT

event_name

- **payload** :
 - **field_name** : type (string | integer | boolean | object | array)

- `routing_derivation` :
 - `owning_node` : If a system node subscribes_ to the event → that node owns it
 - `dual_delivery` : If a node AND agents subscribe → route to both
 - `consuming` : If only a node subscribes → route to node only
 - `passthrough` : If no node subscribes → pure agent delivery

persistence_model

- `description` : Agents do not have raw database access. The platform provides typed persistence tools auto-generated from the `entity_schema` in `package.yaml`. For each field group in the entity schema, the platform generates save/get tools with validated input schemas. This ensures all data access is permissioned, auditable, and storage-backend agnostic.
- `auto_generated_tools` :
 - `get_entity` : Read entity by ID. Returns all fields the agent has permission to see.
 - `save_entity_field` : Write a specific field on an entity. Field must exist in `entity_schema`.
 - `search_entities` : Query entities by stage, field values, or metadata.
 - `query_metrics` : Read aggregated metrics (counts, sums, averages) across entities.
- `schema_source` : `package.yaml` → `entity_schema`

required_agents

- `description` : `required_agents` in `schema.yaml` declares agents the flow needs.
- `fulfillment_rule` : The agent YAML map key must match the role field in `required_agents`. This is the canonical identity: if schema says `role: classifier-agent`, `agents.yaml` must have a top-level key `classifier-agent`. No separate `fulfills` field.
- `note_on_empty` : `required_agents` entries with empty subscriptions and emits are valid. They declare that the flow needs this agent role to exist, even if the agent only uses universal tools (`agent_message`, `mailbox_send`). Common for managerial roles that coordinate via messages rather than event subscriptions.

hello_world_example

- `description` : Minimal flow that processes one event and advances state.

package_yaml

```
name: hello
version: 1.0.0
platform_version: ">=1.1.0"
flows: []
```

schema_yaml

```
initial_state: waiting
terminal_states: [done]
states: [waiting, done]
pins:
  inputs:
    events: [hello.requested]
  outputs:
    events: [hello.completed]
```

nodes_yaml

```
hello-node:
  id: hello-node
  execution_type: system_node
  subscribes_to: [hello.requested]
  produces: [hello.completed]
  event_handlers:
    hello.requested:
      advances_to: done
      emits: hello.completed
```

events_yaml

```
hello.requested:
  payload:
    entity_id: string
    message: string
hello.completed:
```

```
payload:
  entity_id: string
```

- `agents_yaml` : {}
- `produces_derivation` : The produces field on system nodes is OPTIONAL. If omitted, the platform derives it from handler emits, rules emits, on_complete emits, and fan_out emit_per_item at boot. If present, the verifier checks it matches actual emits (PRODUCES-DRIFT warning). Authors should omit produces and let the platform derive it.

underscore_prefix_convention

- `description` : Fields starting with _ in contract YAML files.
- `semantic_fields` :
 - `_source` : Event producer type. Suppresses NO-PRODUCER verifier warnings. Values: external (...)
 - `_consumer` : Event consumer type. Suppresses NO-CONSUMER warnings. Values: mailbox_system (...)
 - `_status` : Event lifecycle. Values: planned (future feature).
 - `_producer` : Event producer agent. Suppresses NO-PRODUCER warnings for agent-emitted events.
- `documentation_fields` : Any other _-prefixed field (_note, _routing_note, _internal_events_note, etc.) is a human comment. The platform ignores them. The verifier ignores them. They are not part of the contract.

handler_specification

Unified handler specification. Each handler field is documented once with its schema, purpose, execution position in the dependency graph, and dependencies.

dependency_graph

- `description` : Arrows mean “must complete before”. Steps at the same level are independent — the implementer may execute them in any order.

graph

```

query (optional – pre-fetch cross-entity data into handler context)
  ↳ clear_gates (optional – reset gates before guard)
    ↳ guard
      ↳ accumulate
        ↳ filter (optional – prune accumulated items)
          ↳ reduce (optional – aggregate filtered items to single value)
            ↳ count (optional – count items)
              ↳ compute
                ↳ on_complete / rules (mutually exclusive)
                  ↳ advances_to ⊃
                    sets_gate ———| (independent of each other)
                    data_accumulation ⊂
                      ↳ payload_transform
                        ↳ emits
                          ↳ action
                            ↳ clear (optional – reset
accumulator/state buckets)

```

atomicity

- **description** : ALL side effects from one handler execution commit in a single database transaction. State advance, gate set, data write, and event persistence happen atomically. No external observer sees intermediate state.
- **consequence** : Since independent steps (advances_to, sets_gate, data_accumulation) commit together, their relative execution order is cosmetic. The database sees one atomic write. An implementer may write data before or after state advance — the result is identical.
- **guards_see_pre_handler_state** : Guards evaluate against entity state BEFORE any handler writes. data_accumulation from the current handler does NOT affect the current handler's guard. It affects the NEXT handler's guard (on the next event, after this transaction commits).

on_complete_vs_rules

- **description** : Mutually exclusive. A handler uses on_complete OR rules, never both.
- **on_complete** : Ordered list of {condition, advances_to, emits} — first match wins. Used after accumulation/computation.

- `rules` : Map of {condition, emits/advances_to/data_accumulation} — payload-based routing. Used for type dispatching.

short_circuits

- `guard_fail` : If guard fails, handler stops. `on_fail` action executes (reject/kill/discard/escalate). No further steps.
- `accumulate_incomplete` : If accumulation is not yet complete, handler stops after recording the arrival. No further steps until completion condition met.

handler_fields

- `description` : Fields within an event_handler declaration. Executed per the dependency graph defined in `handler_execution_order`. All fields are optional — the engine skips absent fields.

DESCRIPTION_FIELD

- `field` : description
- `type` : string
- `purpose` : Human-readable description of what this handler does. Not executed.

GUARD

- `field` : guard
- `type` : object — single check OR list of checks
- `purpose` : Evaluate conditions before proceeding. All checks must pass.
- `single_form` :
 - `id` : string — check identifier
 - `check` : string — expression evaluated against entity state, payload, and policy
 - `policy_ref` : string — optional annotation documenting which policy key is referenced
 - `platform_builtin` : string — optional reference to a platform-provided guard (e.g., authorization checks)
- `list_form` :
 - `checks` : list of {id, check} objects — ALL must pass

- `on_fail` : reject | kill | discard | escalate:{event_name} — what to do when any check fails.
Default: reject.

ACCUMULATE

- `field` : accumulate
- `type` : object
- `purpose` : Track multiple event arrivals for same entity. Wait until completion.
- `sub_fields` :
 - `expected_from` : string — entity field containing expected count or items list
 - `completion` : all | threshold | timeout
 - `from` : string — optional sender path filter
 - `on_complete` : handler fields to execute when done
 - `on_timeout` : handler fields to execute on timeout
 - `description` : string — human-readable note
 - `dedup_by` : string — field path for deduplication. Default: “sender” (sender session ID). Set to a payload field path (e.g., “payload.dimension”) when accumulating items identified by content rather than sender. Duplicate arrivals with the same dedup key are ignored.
- `concept` : Track multiple events arriving for the same entity. Fire `on_complete` when completion condition is met. Expected count read from entity state (written by initiating handler).

COMPUTE

- `field` : compute
- `type` : object
- `purpose` : Execute a platform computation primitive.
- `sub_fields` :
 - `operation` : string — weighted_average | pick_or_average | sum | min | max | count
 - `tiers` : list — for weighted_average: [{dimensions, weight}]
 - `params` : object — operation-specific parameters
 - `store_as` : string — entity field to store result
 - `description` : string — human-readable note

ON_COMPLETE

- `field` : on_complete

- **type** : list of {condition, advances_to, emits} objects — evaluated top-to-bottom, first match wins
- **purpose** : Conditional branching after accumulation or computation.
- **branch_fields** :
 - **condition** : string — expression evaluated against entity state and policy
 - **advances_to** : string — target state
 - **emits** : string — event to emit
- **_note** : MUST be a YAML list (ordered), not a map (unordered). Each entry has a condition evaluated as CEL. First matching condition executes.

ADVANCES_TO

- **field** : advances_to
- **type** : string
- **purpose** : Set entity.state to target. Skipped if on_complete already advanced.

SETS_GATE

- **field** : sets_gate
- **type** : string
- **purpose** : Set entity.gates.{name} = true.

DATA_ACCUMULATION

- **field** : data_accumulation
- **type** : object
- **purpose** : Write event payload fields to entity.
- **sub_fields** :
 - **writes** : list — field names (direct mapping: payload.X → entity.X) OR objects ({source_field: X, target_field: Y} for rename mapping) OR objects ({source_field: X, value: V} for literal value)
 - **source_event** : string — which event the payload comes from (default: current event).
CANONICAL name — use this, not “source”.

canonical_shape

- **description** : One canonical shape. This is the only accepted shape.

canonical

format

```

data_accumulation:
  writes:
    - field_name           # direct: payload.field_name → entity.field_name
    - source_field: payload_key # mapped: payload.payload_key → entity.target_key
      target_field: entity_key
    source_event: event_name # optional: defaults to the trigger event

```

rules

- `writes` is always a list
- each item is either a string (direct name match) or a `{source_field, target_field}` object (mapped)
- `source_event` is optional – if omitted, the trigger event is the source
- the `source_event` payload must contain all referenced fields (verified at boot)

format

```

data_accumulation:
  writes:
    - field_name           # direct: payload.field_name → entity.field_name
    - source_field: payload_key # mapped: payload.payload_key → entity.target_key
      target_field: entity_key
    - target_field: entity_key # computed: CEL expression → entity.target_key
      expression: "entity.counter + 1"
    source_event: event_name # optional: defaults to the trigger event

```

rules

- computed writes use `{target_field, expression}` – expression is CEL evaluated against handler context (entity, payload, policy)
- `source_event` is optional. When omitted, the trigger event is the source. When present, it must be a valid event name in `events.yaml`. It is NOT a path expression – use `data_accumulation.writes` with `{source_field, target_field}` for field mapping.

EMITS

- `field` : emits
- `type` : string or list of strings

- `purpose` : Publish follow-up event(s). Skipped if `on_complete` already emitted.

RULES

- `field` : rules
- `type` : map of `rule_name` → rule
- `purpose` : Type-based routing. Match payload field against conditions.
- `rule_fields` :
 - `condition` : string — expression evaluated against payload
 - `emits` : string or list — event(s) to emit
 - `advances_to` : string — optional state change
 - `data_accumulation` : object — optional data write (same format as handler-level)
 - `description` : string — human-readable note

FAN_OUT

- `field` : `fan_out`
- `type` : object
- `purpose` : For each item in a list, emit an event.
- `sub_fields` :
 - `items_from` : string — source list (payload field or entity field)
 - `target` : string — agent path to receive events
 - `emit_per_item` : string — event name to emit for each item (all same type)
 - `emit_mapping` : object — optional: map item key values to different event names
- `side_effect` : Writes `fan_out.count` to handler context (available for `data_accumulation`).

QUERY

- `field` : query
- `type` : object or list of objects
- `purpose` : Cross-entity read and aggregation.
- `sub_fields` :
 - `entities` : string — entity table name
 - `filter` : string — optional condition

- `group_by` : string — field to group by
- `count` : boolean — count per group
- `select` : list — fields to include
- `store_as` : string — where to put results (in emit payload)
- `execution_position` : Before `clear_gates`. Pre-fetches cross-entity data into handler context for use by `guard`, `compute`, or `on_complete`.

REDUCE

- `field` : reduce
- `type` : object
- `purpose` : Combine accumulated items into a single value.
- `sub_fields` :
 - `items_from` : string — source list
 - `operation` : string — `weighted_average` | `pick_or_average` | `sum` | `min` | `max` | `count`
 - `params` : object — operation-specific parameters
 - `store_as` : string — entity field to store result
- `execution_position` : After `filter`, before `count`. Aggregates filtered items into a single value (`weighted_average`, `sum`, `min`, `max`, `pick_or_average`).

FILTER

- `field` : filter
- `type` : object
- `purpose` : Keep items from a list that match a condition.
- `sub_fields` :
 - `items_from` : string — source list
 - `condition` : string — expression per item
 - `store_as` : string — entity field for result
- `execution_position` : After `accumulate`, before `reduce`. Prunes accumulated items based on CEL condition. Only matching items pass through.

COUNT

- `field` : count
- `type` : object
- `purpose` : Count items matching a condition.
- `sub_fields` :
 - `items_from` : string — source list
 - `condition` : string — optional filter
 - `store_as` : string — entity field for count
- `execution_position` : After reduce, before compute. Counts items (optionally matching a condition). Stores result in entity field.

CLEAR

- `field` : clear
- `type` : object
- `purpose` : Bulk reset state tables.
- `sub_fields` :
 - `targets` : list of state store names to clear
- `execution_position` : After action. Last step. Resets accumulator state or specific entity field buckets. Distinct from `clear_gates` which runs before guard.

ACTION

- `field` : action
- `type` : string
- `purpose` : Names a platform-provided action. Valid values: `create_flow_instance`, `record_evidence`. The platform executes the named action. NOT for platform actions — all behavior must be declarative.

valid_values

create_flow_instance

- `description` : Create a new dynamic flow instance.
- `required_sibling_fields` :
 - `template` : string — flow template ID (must be mode: template)

- `instance_id_from` : string — payload field containing the unique instance ID
- `config_from` : object — maps instance variable names to payload field paths. Platform reads payload values and passes them as config to the new flow instance.

record_evidence

- `description` : Append event payload to entity accumulator state.
- `behavior` : Reads the event payload and appends it as a new entry in the entity's evidence accumulator (from `state_schema`). Append-only — never replaces. Used for building an audit trail of signals, scores, or decisions.
- `required_sibling_fields` :

PAYLOAD_TRANSFORM

- `field` : payload_transform
- `type` : object
- `purpose` : Construct an output event payload from multiple sources. Used when emitting an event that combines fields from entity state, incoming event payload, and computed values.
- `sub_fields` :
 - `fields` : map of `output_field_name` → source expression (CEL)

EXAMPLE

```
payload_transform:
  fields:
    vertical_id: payload.vertical_id
    brand_choice: entity.brand
    mandate: entity.mvp_spec
    founder_directives: payload.notes
```

CLEAR_GATES

- `field` : clear_gates
- `type` : boolean
- `purpose` : Reset all entity gates to false. Used before re-entering a validation cycle.
- `scope` : Clears all gates on the ENTITY, not scoped to flow schema or node. An entity has one flat gate map. `clear_gates: true` resets every key in that map to false. Rationale: gates are entity-level state. A flow that resets an entity to an earlier phase (e.g., `vertical.needs_more_data` returning to researching) must clear all gates, not just the ones its node declared.
- `limitation` : All-or-nothing. Selective gate clearing (`clear_gates: [gate_a]`) is a potential v1.3 feature.

EVIDENCE_TARGET

- `description` : Required when action is `record_evidence`. Specifies which accumulator key the evidence is appended to.
- `type` : string
- `example` : `build_evidence`

node_specification

NODE_FIELDS

- `description` : Top-level fields on a system node declaration.
- `required` :
 - `id` : string — unique node identifier within the flow
 - `execution_type` : string — must be “`system_node`”
 - `subscribes_to` : list of event names this node receives
 - `event_handlers` : map of `event_name` → handler declaration
- `optional` :
 - `description` : string — human-readable description
 - `state_schema` : map — fields the node persists per entity (accumulator state)
 - `state_table` : string — table name for node state persistence
 - `timers` : list of timer declarations (see `timer_model`)
 - `gate_state` : map — gate field definitions for this node (boolean flags on entity)
 - `permissions` : list of platform permissions this node requires (e.g., `create_flow_instance`)
- `description` : Complete specification for system nodes — the deterministic components that orchestrate state transitions. Every field used in `nodes.yaml` must be defined here.

dependency_rules

- `query → clear_gates`: cross-entity data fetched before any state evaluation
- `clear_gates → guard`: gates reset before guard reads them
- `guard → accumulate`: guard must pass before accumulation is checked
- `accumulate → filter`: accumulated items available before pruning
- `filter → reduce`: pruned items available before aggregation

- `reduce` → `count`: aggregated result available before counting
- `count` → `compute`: all list processing complete before computation
- `compute` → `on_complete/rules`: computed values written before branch conditions evaluate
- `on_complete/rules` → `{advances_to, sets_gate, data_accumulation}`: branches may override these fields
- `advances_to, sets_gate, data_accumulation`: INDEPENDENT – no causal dependency, any order valid
- `payload_transform` → `emits`: output payload constructed before event persisted
- `emits` → `action`: events persisted before platform actions execute
- `action` → `clear`: state cleanup runs last, after all side effects committed

core_vs_extensions

- `description`: Most handlers use a small core subset. Advanced features are extensions.

CORE_FIELDS

- `description`: Used by 90%+ of handlers. Learn these first.

fields

- `guard`
- `advances_to`
- `sets_gate`
- `data_accumulation`
- `emits`
- `rules`
- `on_complete`

EXTENSION_FIELDS

- `description`: Used by specialized handlers. Learn when needed.

fields

- `accumulate`
- `compute`
- `fan_out`

- `filter`
- `reduce`
- `count`
- `query`
- `clear`
- `payload_transform`
- `clear_gates`
- `action`
- `yaml_field_order` : YAML field order within a handler is cosmetic. The engine executes fields per the dependency graph, not per source order. A handler may list `on_complete` before `accumulate` in YAML — the engine still runs `accumulate` first. Authors SHOULD order fields to match the dependency graph for readability, but the engine does not require it.

rules_handler_interaction

- `description` : When a handler has both rules and handler-level fields (`emits`, `data_accumulation`).
- `execution_order` : 1. rules execute first — the matching rule fires its condition-specific side effects (rule-level `emits`, rule-level `data_accumulation`, rule-level `advances_to`). 2. Handler-level `data_accumulation` executes AFTER rules — it supplements rule-level writes, does not replace them. Both write to the entity in the same atomic transaction. 3. Handler-level `emits` executes AFTER rules — rule-level `emits` AND handler-level `emits` BOTH fire. Handler-level `emits` are unconditional; rule-level `emits` are conditional on the matched rule.
- `example` : `vertical.discovered` has handler-level `data_accumulation` (6 fields) + rules with per-rule `data_accumulation` (`scoring_rubric`). Result: all 6 handler-level fields are written, PLUS the matching rule writes `scoring_rubric`. Handler-level `emits` (`scoring.requested`) fires regardless of which rule matched.
- `precedence` : Handler-level `advances_to`, `sets_gate`, `data_accumulation`, and `emits` are defaults. If the matched rule specifies any of these, the rule value OVERRIDES the handler-level value for that field only. Fields not specified in the rule fall through to handler-level.

engine

Specification for the platform engine — how it loads, validates, and executes flows.

flow_tree_walker

- `description` : Recursive discovery of flows from the root `package.yaml`. The walker collects all flows into a flat registry. Validation happens after collection.

ALGORITHM

- 1. Start at root directory. Read `package.yaml`.
- 2. Register this flow: record its directory, package manifest, and all contract files present.
- 3. For each entry in `package.yaml` flows: list, resolve child directory: `{current_dir}/flows/{child.flow}`.
- 4. If child directory exists and contains `package.yaml`, recurse from step 1 with child directory.
- 5. If child directory missing or no `package.yaml`, log warning and skip.
- 6. Track visited directories. If already visited, skip (prevents infinite loops from duplicated copies).
- 7. Stop when depth exceeds 99. Log: "Flow nesting depth exceeded 99 – contact platform team."

MINIMUM_REQUIRED_FILES

always_required

- `package.yaml`
- `schema.yaml`
- `at_least_one` : `agents.yaml` OR `flows/` with at least one child
- `boot_error` : A flow with no `agents.yaml` and no child flows is an error.
- `optional_files` :
 - `nodes.yaml` : System node handlers. Omit if flow only composes children via agents.
 - `events.yaml` : Event payload schemas. Omit if flow uses only events declared by children or root.
 - `tools.yaml` : Flow-specific tools. Omit if flow uses only root-level or platform tools.

- `policy.yaml` : Flow-specific thresholds. Omit if flow uses only root-level policy.
- `prompts/` : Agent prompts. Required if `agents.yaml` exists.
- `output` : A flat list of `FlowEntry` records. Each record contains: flow ID, directory path, depth, parent flow ID, package manifest, and paths to all contract files found. This flat list is the input to the contract merger (next step).
- `max_depth` : 99
- `duplicate_handling` : Flows may be duplicated (same flow directory copied into multiple parents). Each copy is a separate flow instance. No deduplication. No DAG resolution. The walker treats each directory as independent.
- `flow_id_uniqueness` : Flow IDs (from `package.yaml` flows: list) must be unique within the entire tree. Duplicate flow IDs across different parents is a boot error.

uri_addressing

- `description` : Every addressable runtime entity (node, agent, event) is identified by a path. Paths follow a simple hierarchical model: slash means navigating into a child flow. No slash means local to the current flow.

RULES

local

- `format` : {name}
- `meaning` : Entity in the current flow instance.

examples

- `scoring.requested`
- `analysis-agent`
- `scoring-node`

absolute

- `format` : {flow_id}/{flow_id}/.../{name}
- `meaning` : Entity in a specific flow instance. Path navigates from root.

examples

- `scoring/entity.shortlisted – scoring flow's event`

- `validation/research/research.completed` – nested sub-flow event
- `operating/opco-ceo` – agent in operating flow
- `portfolio-node` – root flow's node (no slash = root-local)
- **resolution** : Slash presence determines scope. No slash = local to current flow. Slash present = absolute path from root. Local names never contain slashes. The platform resolves all references to full internal paths at boot.
- **type_inference** : No type suffix needed. The context determines the type: subscriptions and `emit_events` contain events. Message targets contain agents. `subscribes_to` in nodes contains events. The type is implicit from where the reference appears.
- **full_uri** :
 - **description** : For cross-root references (future — multiple root flows on one platform), full URI format is available: `{root}://{path}/{name}`. Within a single root flow, absolute paths are sufficient.
 - **format** : `{root}://{path}/{name}`
 - **example** : `empire://scoring/entity.shortlisted`
 - **when** : Only needed when multiple root flows run on the same platform instance.

WILDCARDS

- **description** : Pattern matching for subscriptions across flow instances.
- **patterns** :
 - `*/entity.shortlisted` : Any direct child flow's `entity.shortlisted` event
 - `**/entity.completed` : Any flow at any depth

contract_merger

- **description** : After the flow tree walker collects all flows, the merger builds the runtime registry. Every node, agent, and event gets a full URI. No global merge — everything is scoped by flow instance.

PROCESS

- 1. For each flow in the flat list (from walker), construct its instance path.
- 2. For each node in flow's `nodes.yaml`: register with URI `{root}://{instance_path}/{node_id}`

- **3.** For each agent in flow's `agents.yaml`: register with URI `{root}://{instance_path}/{agent_id}`
- **4.** For each event in flow's `events.yaml`: register with URI `{root}://{instance_path}/{event_name}`
- **5.** For each tool in flow's `tools.yaml`: register as shared (tools are not URI-scoped)
- **6.** For each policy key in flow's `policy.yaml`: register scoped to flow instance
- **7.** Root flow's tools and policy are inherited by all children unless overridden.
- **no_conflicts** : URI scoping eliminates all conflicts. Two flows with a node called “orchestrator” become `empire://discovery/orchestrator` and `empire://validation/orchestrator`. No merge logic needed. No collision possible.
- **subscription_resolution** : After all URIs are constructed, resolve agent subscriptions. Local subscriptions (no `://` prefix) resolve within the same flow instance. Full URI subscriptions resolve globally. Wildcard subscriptions expand to matching URIs.
- **tools** : Tools are NOT URI-scoped. They are shared resources. Root flow tools are available to all child flows. Child flow tools are available only within that flow. If a child declares a tool with the same ID as a root tool, the child’s version takes precedence within that flow.
- **policy** : Policy keys are scoped by flow. Root flow policy is inherited by all children. Child flow policy overrides root for the same key within that child. Guards and prompt templates resolve policy keys by walking up the flow tree: `child → parent → root`.
- **output** : A runtime registry containing: `URI → Node map`, `URI → Agent map`, `URI → Event schema map`, tool registry (flat, with flow-level overrides), policy tree (hierarchical, child overrides parent).

agent_sharding

- **description** : Parallelism within a flow is handled by system nodes spawning multiple agent instances. The platform provides the mechanics (`agent_hire`, event routing, accumulation tracking). The product node decides the partitioning logic.
- **model** : A system node handler receives a trigger event with a workload (e.g., 52 subcategories). The node partitions the workload and spawns `N` agent sessions, one per partition. Each agent receives a partition-specific event. The node tracks completion via the `accumulate` primitive. When all partitions complete, the node proceeds.

PLATFORM_PROVIDES

- `agent_hire` tool – spawn an agent session with specific configuration
- Event routing – deliver partition-specific events to the correct agent instance
- Accumulation tracking – track which partitions have completed
- Timeout – fire `on_timeout` if partitions don't complete in time

PRODUCT_PROVIDES

- Partitioning logic – how to split the workload
- Partition event construction – what each agent instance receives
- Merge/reduce logic – how to combine partition results (if needed)
- `not_in_scope` : Flow-level sharding (distributing flow instances across compute nodes) is not supported in v1.0.0. All flow instances run on the same platform instance. Agent sharding provides within-flow parallelism. Cross-machine distribution is a future concern.
- `addressing` : Sharded agent instances are addressed by their agent session ID, not by a special shard path. The wildcard `*` in paths matches dynamic flow instances, not agent shards. Agent shards are ephemeral — they exist for the duration of the partition work and are not addressable after completion.

dynamic_instance_lifecycle

- `description` : Flow instances can be created at runtime by system nodes calling `create_flow_instance`. The flow template must be declared as `mode: template` in the parent `package.yaml`.

CREATION

- `trigger` : A system node handler calls `create_flow_instance` (platform builtin tool). The node decides when to create instances based on its product logic. Same pattern as agent sharding — nodes orchestrate instance creation.
- `instance_id` : Supplied by the node (from event payload or generated). Must be unique within the template scope. If already exists, creation fails with error.

process

- 1. Node handler calls `create_flow_instance` with template ID and `instance_id`
- 2. Platform validates: template exists and is `mode: template`
- 3. Platform validates: `instance_id` is unique within template scope

- 4. Platform loads flow template contracts from `flows/{template}/`
- 5. Platform constructs paths: `{template_id}/{instance_id}/{local_name}`
- 6. Platform registers new nodes, agents, events in runtime registry
- 7. Platform expands wildcard subscriptions to include new instance
- 8. Platform resolves local subscriptions for new instance
- 9. Platform creates entity record at `initial_state` from `schema.yaml`
- 10. New instance nodes and agents start
- **auto_emit**: Flow schemas may declare `auto_emit_on_create`. When the platform creates an instance, it automatically emits the declared event after the instance is fully started. This bootstraps the flow without requiring an external trigger.
- **addressing**:
 - **static_flows**: `{flow_id}/{local_name}` — one level, no instance segment
 - **template_instances**: `{template_id}/{instance_id}/{local_name}` — extra segment for instance
 - **wildcard**: `{template_id}/*/ {event_name}` — matches any instance under this template
- **wildcard_subscriptions**:
 - **description**: Agents that need to observe all dynamic instances use wildcard paths. When a new instance is created, existing wildcard subscriptions expand to include it.
 - **static_vs_dynamic**: Static flows are declared in `package.yaml` with `mode: static`. Template instances are created at runtime with `mode: template`. Wildcard `*` only matches template instances. Static flow names are always explicit.
- **termination**:
 - **terminal_state**: A dynamic instance reaches a terminal state from `schema.yaml`. Instance stops processing events. State preserved for querying. Agents and nodes stopped.
 - **cleanup**: Operational concern. Platform does not auto-delete. Future: TTL or explicit destroy tool.

boot_sequence

- **description**: Ordered steps from platform start to ready. If any validation step fails, boot aborts with a clear error. No partial startup.

STEPS

1. **step** : 1; **name** : load_platform_spec; **action** : Read platform/contracts/platform-spec.yaml. Verify platform version.
 2. **step** : 2; **name** : walk_flow_tree; **action** : Starting from root package.yaml, recursively discover all flows. Build flat list of FlowEntry records. Max depth 99.
 3. **step** : 3; **name** : construct_paths; **action** : For each flow, construct hierarchical paths for its nodes, agents, and events. Local names become absolute paths: {flow_instance_path}/{local_name}.
 4. **step** : 4; **name** : register_templates; **action** : For flows declared as mode: template, register the template but create no instances. The template is available for runtime create_flow_instance calls.
 5. **step** : 5; **name** : build_registries; **action** : Build runtime registries keyed by path: nodes, agents, events, tools, policy. Tools are flat (shared). Policy is hierarchical (child overrides parent).
 6. **step** : 6; **name** : resolve_subscriptions; **action** : Resolve all agent and node subscriptions. Local names (no /) resolve within same flow. Paths with / resolve as absolute from root. Wildcards expand to matching paths.
 7. **step** : 7; **name** : validate_pins; **action** : For each flow: required input event pins must be wired (some other flow emits them). No two flows may write the same entity field (write conflict = boot error).
 8. **step** : 8; **name** : validate_required_agents; **action** : Every flow's required_agents roles must be fulfilled by agents in that flow's agents.yaml.
 9. **step** : 9; **name** : validate_tools; **action** : Every tool in every agent's tools_tier2 must exist in the tool registry.
 10. **step** : 10; **name** : validate_permissions; **action** : Every agent has sufficient permissions for its declared tools.
 11. **step** : 11; **name** : validate_platform_version; **action** : Root flow's platform_version range includes the running platform version.
 12. **step** : 12; **name** : initialize_state_stores; **action** : Create or verify entity tables, accumulator state tables, event store. Derive schema from package.yaml entity_schema and nodes.yaml state_schema.
 13. **step** : 13; **name** : start_system_nodes; **action** : Each node subscribes to its declared events. Nodes are ready to handle.
 14. **step** : 14; **name** : start_agents; **action** : Each agent's subscriptions are active. Agent sessions created on first event.
 15. **step** : 15; **name** : ready; **action** : Platform is accepting events. Log boot summary: flow count, node count, agent count, event count.
- **failure_handling** : Any step that fails aborts boot with a clear error message identifying: which step failed, which flow/agent/event caused the failure, and what the expected fix is. No partial startup — either everything validates or nothing runs.

event_loop

- **description** : The event loop is the core runtime cycle. Events enter, get persisted, routed to subscribers, and processed. This runs continuously after boot.

LIFECYCLE

1. **step** : 1; **name** : event_arrives; **action** : An event enters the system. Sources: agent emit, system node emit, timer fire, human input, handoff trigger, external API.
 2. **step** : 2; **name** : validate_payload; **action** : Validate event payload against the schema in events.yaml. Reject if schema mismatch. Log and discard invalid events.
 3. **step** : 3; **name** : persist_event; **action** : Write event to the event store (write-ahead). The event is durable before any processing begins. Crash after this point: event replays on recovery.
 4. **step** : 4; **name** : resolve_subscribers; **action** : Look up subscribers by event path. Two kinds: system nodes (from nodes.yaml subscribes_to) and agents (from agents.yaml subscriptions). Wildcard subscriptions are pre-expanded at boot and updated on dynamic instance creation.
 5. **step** : 5; **name** : node_handling; **action** : For each subscribing system node: acquire entity lock, execute the dependency-graph handler within an atomic boundary, release entity lock. New events emitted by the handler are persisted within the same atomic boundary but delivered AFTER commit (step 7).
 6. **step** : 6; **name** : agent_delivery; **action** : For each subscribing agent: place the event in the agent's inbox. Agent processes it asynchronously. No lock, no blocking.
 7. **step** : 7; **name** : deliver_emitted_events; **action** : Events emitted by node handlers in step 5 are now delivered. Each enters the event loop at step 1 in its own cycle. This prevents recursive handler chains and deadlocks.
 8. **step** : 8; **name** : mark_delivered; **action** : Original event marked as delivered in the event store.
- **concurrency** :
 - **description** : Events are serialized per entity, concurrent across entities.
 - **per_entity_serialization** : All events for the same entity are processed one at a time. When a handler is processing an event for entity X, any other event for entity X waits until the handler completes and commits. This prevents race conditions on entity state, accumulator, and gates.
 - **cross_entity_concurrency** : Events for different entities are fully concurrent. Handling entity X does not block entity Y. The platform can process thousands of entities simultaneously.
 - **entity_lock** : The platform acquires an entity-level lock before handler execution. Lock scope: one entity, one handler. Released after commit or rollback. Lock mechanism is implementation-specific (DB row lock, in-memory mutex, etc.).

- **agent_concurrency** : Agent deliveries are asynchronous. Events placed in agent inbox in order. Agent processes inbox events sequentially. Multiple agents run concurrently. No entity lock needed for agent processing — agents interact with entities through platform tools which acquire their own locks.

ATOMICITY

- **description** : All side effects of one handler execution are atomic. Either everything commits or nothing does.

within_atomic_boundary

- **Entity state change (advances_to)**
- **Gate updates (sets_gate)**
- **Accumulator state updates (data_accumulation, accumulate tracking)**
- **New events persisted to event store (emits)**

outside_atomic_boundary

- **Delivery of emitted events to their subscribers (happens after commit)**
- **Agent inbox delivery (async)**
- **failure** : If any write within the atomic boundary fails, all writes roll back. The triggering event is marked as failed. Failed events can be retried (handler is idempotent).
- **crash_recovery** :
 - **description** : On restart, the platform replays undelivered events. Events persisted (step 3) but not marked delivered (step 8) are replayed. Handler idempotency ensures replayed events produce the same result.
- **single_node_handler_rule** :
 - **description** : Each event may be handled by at most ONE system node. If two system nodes declare the same event in their event_handlers, boot fails. This ensures unambiguous state authority — one event, one state transition, one owner.
 - **agents** : Not constrained. Multiple agents may subscribe to the same event freely.
 - **enforcement** : Boot validation rejects duplicate node handlers for the same event.
- **emit_payload_default** : When a handler emits without payload_transform, the platform constructs the payload by: 1. Start with contract-visible entity fields (from entity_state.fields). 2. Overlay triggering event payload (trigger wins on collision with entity fields). 3. Force platform fields: entity_id,

trigger_event_type, current_state (always present, always from platform). 4. If payload_transform is present, its result replaces steps 1-2 entirely (step 3 still applies). Collision order: platform fields > payload_transform > trigger payload > entity fields.

- **dual_path_events** : An event can be both an input pin (arriving from outside the flow) and internally generated (emitted by a handler within the flow). The system node handler processes it identically regardless of source. No priority or deduplication between paths — if the same event fires from both, it runs twice (idempotency_key in the events table prevents true duplicates).

state_management

- **description** : The platform tracks three kinds of state per entity instance. All state is persisted. All mutations happen within the handler atomic boundary.
- **entity_state** :
 - **description** : A single string value representing where the entity is in the flow lifecycle. Set by advances_to in handler execution. Initial value from schema.yaml initial_state. Terminal values from schema.yaml terminal_states.
 - **storage** : Entity table, state column, indexed.
 - **mutation** : Only via advances_to or on_complete branch. Never by agents directly.
- **accumulator_state** :
 - **description** : Structured data tracking partial progress within a handler's accumulate pattern. Examples: which scoring dimensions have arrived, which scanner partitions completed, intermediate computed values.
 - **storage** : Per-node state table (from nodes.yaml state_schema). Keyed by entity_id.
 - **mutation** : Written by accumulate tracking (record arrival), data_accumulation (write payload fields), and compute steps (store computed values). Cleared or reset when accumulation completes.
- **gate_state** :
 - **description** : Boolean flags on the entity. Used by validation-style flows to track which sub-steps have completed. Guards can check gate state as conditions.
 - **storage** : Entity table, gates column (jsonb or equivalent).
 - **mutation** : Set via sets_gate handler field. Default behavior: set to true, never reset. If an entity needs to re-enter a validation cycle, use clear_gates handler field to reset all gates to false before re-entering.

- **scoping** : All state is scoped by entity_id + flow instance path. Two entities in the same flow have independent state. Two instances of the same flow template have independent state. There is no shared mutable state between entities or between flow instances.
- **querying** : State is queryable at any time without side effects. The platform may provide a query_flow_state tool for agents to read entity state, accumulator, and gates without emitting events.

TERMINAL_STATE_BEHAVIOR

- **description** : When an entity reaches a terminal state (from schema.yaml terminal_states), all state is preserved. Entity state, accumulator state, gate state, and event history remain queryable indefinitely.
- **no_cleanup** : The platform does NOT clear entity data on terminal state. Killed verticals retain all scores, signals, research briefs, and decision history for post-mortem analysis and pattern learning.

behavior

- Entity stops accepting new events (handlers reject events for terminal entities)
- Timers are cancelled
- Agent sessions are terminated
- All persisted state remains in the database
- Entity is queryable via query_entities tool
- **archival** : Future concern — retention policy and cold storage are not specified in v1.1.0.
- **event_rejection** : When an event arrives targeting an entity in a terminal state, the platform rejects it before handler execution. No guard runs, no handler runs, no state changes. The event is marked as rejected with reason: terminal_entity. This is unconditional — no configuration can override it. Terminal means terminal.
- **absorbing** : Terminal states are absorbing by default. Once an entity reaches a terminal state, it cannot leave. All events for that entity are rejected. This is unconditional.
- **no_reopen_mechanism** : There is no opt-in mechanism to reopen a terminal entity. If a business process needs to “reopen” a killed/closed entity, the correct pattern is to create a NEW entity (new entity_id) with data copied from the old one. The old entity remains terminal. This preserves audit trail integrity.
- **backward_transitions** : Backward transitions (e.g., cto_review → mvp_speccing) are allowed for NON-terminal states. The prohibition is specifically on leaving terminal states, not on backward movement in general. Revision loops, reset-to-researching, and similar patterns are valid.

- **entity_field_resolution** :
 - **description** : In CEL expressions, `entity.X` resolves against a unified namespace that includes: `entity_state.fields` (from `data_accumulation` writes), `entity_state.current_state` (as `entity.state`), `entity_state.gates` (as `entity.gates.X`), and `node state_schema` fields (from `accumulate/compute` writes). There is no distinction between `entity_schema` and `state_schema` at resolution time — both are accessible via `entity.X`. The storage layer may use separate tables, but the CEL context merges them.
- **state_schema_scoping** : `state_schema` fields are node-scoped in storage. In CEL, `entity.X` reads the current handler node state. Cross-node state reads are not supported — use events to pass data between nodes.

accumulation_engine

- **description** : The accumulation engine is the platform subsystem that handles the `accumulate` primitive. It tracks multiple event arrivals for the same entity and fires on completion.
- **tracking** :
 - **setup** : When an entity first encounters a handler with `accumulate.track`, the platform creates an accumulator record: expected items list, received items list (empty).
 - **arrival** : Each event arrival is identified by its `dedup_by` key. Default: sender session ID. Can be overridden to a payload field (e.g., `payload.dimension`). If the key was already received, the arrival is ignored (idempotent). If not, it is added to the received set and persisted.
 - **completion_check** : After each arrival, check the completion condition: `all` (every expected item received), `threshold` (received count $\geq N$), or `timeout` (timer fires regardless of count).
- **completion_modes** :
 - **all** : Proceed when every item in the expected list has been received.
 - **threshold** : Proceed when received count reaches a configured number.
 - **timeout** : Proceed when a timer fires, regardless of how many items arrived. Partial results available.
- **on_complete** : When completion condition is met, the handler continues to the remaining steps (`compute`, `on_complete` branches, `advances_to`, `emits`). The accumulated data is available to these steps.
- **on_timeout** : If the handler declares `accumulate.on_timeout`, the platform executes that branch when the timer fires before completion. Partial accumulated data is available. Typically used to proceed with whatever data arrived.

- **idempotency** : Duplicate arrivals (same item, same entity) are ignored. The received list is a set, not a list. Crash recovery replays events through the same tracking — already-received items are skipped, accumulator state is consistent.
- **cleanup** : After completion fires and the handler commits, the accumulator record can be archived or deleted. The entity has moved past the accumulation state.

agent_session_management

- **description** : The platform manages agent sessions — the lifecycle of an agent from receiving an event to completing its response.

SESSION_LIFECYCLE

- 1. Event arrives in agent inbox.
- 2. Platform creates agent session: load prompt from prompts/, substitute {{variables}} from policy.
- 3. Platform provides tools (from tools.yaml) to the agent session.
- 4. Agent processes event: reasons, may call tools, may emit events.
- 5. Each tool call is validated (schema check, permission check) before execution.
- 6. Each emitted event is validated (payload schema) and enters the event loop.
- 7. Agent session completes when agent signals done or max_turns_per_task reached.
- 8. Session state is persisted for audit/debug.
- **tool_execution** : Tools execute within the agent session context. Platform validates tool input against schema. Platform checks agent permissions before execution. Tool results are returned to the agent for continued reasoning.
- **event_emission** : When an agent emits an event, the platform validates the payload against events.yaml schema. The event enters the event loop at step 1. Agent emission is NOT within a node handler atomic boundary — it is a standalone event publication.
- **turn_budget** : Each agent session has a max_turns_per_task (from agents.yaml). If the agent exceeds this budget, the session is terminated. The platform may emit a timeout or budget_exceeded event.

- **agent_invocation** : Agents are invoked via `claude -p` (CLI) or equivalent LLM API call. The platform constructs the invocation: system prompt + event context + tool schemas. Agents are short-lived and task-scoped. No persistent memory between sessions. Context from previous interactions is provided via event payload and entity state.

CONVERSATION_MODES

- **description** : How the platform manages agent session context across invocations.
- **modes** :
 - **task** : New session per event. No memory between invocations. Stateless.
 - **session** : Session persists across events for the agent. Context accumulates.
 - **session_per_entity** : One session per entity the agent works on. Agent retains context across multiple events for the same entity but has separate sessions for different entities. Useful for agents that need continuity (e.g., research agent building a brief over multiple turns).
- **default** : task
- **subscription_types** :
 - **subscriptions** : Static subscription list. Always active from boot (or from instance creation for dynamic flows). Cannot be modified at runtime. This is the agent's declared event contract.
- **emit_tool_convention** :
 - **description** : The platform auto-generates emit tools from each agent's `emit_events` list. Agents do NOT need to list emit tools in `tools_tier2`.
 - **naming** : `emit_{event_name}` where dots are replaced with underscores. Example: `emit_events: [score.dimension_complete] → tool: emit_score_dimension_complete`
 - **behavior** : Each emit tool accepts a payload object matching the event's schema in `events.yaml`. The platform validates the payload, attaches sender context, and publishes the event.
 - **universal_tools** : Some tools are auto-granted to all agents without explicit `tools_tier2` listing: `agent_message`, `mailbox_send`. These are "universal" tools.
- **task_mode_stateless** : Agents in task conversation_mode do NOT get an `agent_sessions` row. Each event invocation is independent. No conversation history persisted. Debugging traces for task invocations: `spend_ledger` (cost per invocation) + events (what the agent emitted). Provider session IDs for task invocations are ephemeral and not stored.
- **provider_session_ids** : External LLM provider session IDs (e.g., Anthropic conversation ID) are stored in `agent_sessions.runtime_state.provider_session_id`, NOT in `session_id`. `session_id` is the platform-owned UUID primary key. The provider ID is an implementation detail that may change if the provider is swapped.

- `conversation_mode_values` :
 - `task` : Stateless. No session persisted. Each invocation is independent.
 - `session` : Persistent session across events. Conversation history maintained.
 - `session_per_entity` : One session per agent × entity. Context preserved per entity.
 - `stateless` : Alias for task. Accepted by the loader, normalized to task internally.

timer_model

- `description` : Timers are durable delayed events. The platform schedules them and fires them as regular events entering the event loop. Timers are declared in nodes.yaml.

DECLARATION

- `description` : Timers are declared per system node in nodes.yaml.
- `fields` :
 - `id` : Unique timer name
 - `event` : Event name to fire when timer expires (e.g., timer.scan_timeout)
 - `delay` : Duration string (e.g., 72h, 30m, 7d) or policy reference (e.g., policy.validation_gate_timeout)
 - `recurring` : Boolean — if true, timer re-fires at the interval until cancelled
 - `start_on` : State or event that starts the timer (e.g., state:mvp_specpecting or event:validation.started)
 - `cancel_on` : State or event that cancels the timer (e.g., state:ready_for_review or event:spec.approved)

LIFECYCLE

- 1. Timer starts when the entity enters the `start_on` state or the `start_on` event fires.
- 2. Platform persists the timer (`entity_id`, `timer_id`, `fire_at` timestamp).
- 3. When `fire_at` is reached, platform emits the timer event into the event loop.
- 4. Timer event is a regular event – routed to subscribers, processed by handlers.
- 5. If `cancel_on` state is reached or `cancel_on` event fires before `fire_at`, timer is cancelled.
- 6. Recurring timers restart after firing unless cancelled.

- `crash_recovery` : Timers are persisted. On restart, platform checks for expired timers and fires them.

example

```
timers:
  - id: validation_gate_timeout
    event: timer.validation_timeout
    delay: "{{validation_gate_timeout_hours}}h"
    start_on: state:researching
    cancel_on: state:ready_for_review
    recurring: false
  - id: portfolio_digest
    event: timer.portfolio_digest
    delay: "{{digest_interval_hours}}h"
    start_on: boot
    recurring: true
```

expression_language

- `description` : All guard checks, rule conditions, filter conditions, and `on_complete` branch conditions are evaluated using CEL (Common Expression Language). CEL is a non-Turing-complete, safe expression language designed for policy evaluation.
- `specification` : <https://github.com/google/cel-spec>
- `go_implementation` : <https://github.com/google/cel-go>
- `context_variables` :
 - `entity` : Current entity state (all fields from `entity_schema` + `state` + `gates`)
 - `payload` : Current event payload fields
 - `policy` : Policy values from `flow policy.yaml` merged with `root policy.yaml`
 - `fan_out` : Available after `fan_out` step (e.g., `fan_out.count`)
 - `accumulated` : Available inside `on_complete` after accumulation (list of received items)

SUPPORTED_OPERATORS

- Comparison: `==`, `!=`, `<`, `<=`, `>`, `>=`
- Logical: `&&`, `||`, `!`
- Arithmetic: `+`, `-`, `*`, `/`
- Membership: `in`, `contains`
- String: `startsWith`, `endsWith`, `matches`

- `List: size, exists, all, filter, map`

EXAMPLES

- `entity.generation_depth <= policy.max_derivation_depth`
- `payload.mode == "saas_gap"`
- `entity.scores.icp_crispness >= policy.min_icp_score`
- `entity.red_flag_count == 0`
- `payload.decision == "approve"`
- `entity.revision_count < policy.inner_revision_max`
- `entity.composite_score >= policy.composite_shortlist`

error_model

- `description` : How the platform handles failures at each layer.
- `event_validation_failure` :
 - `behavior` : Reject event. Log error. Do not deliver. No retry.
 - `visibility` : Error logged with event name, payload, and validation error.

HANDLER_EXECUTION_FAILURE

- `behavior` : Rollback atomic boundary. Mark event as failed for this handler.
- `retry_policy` :
 - `max_retries` : 3
 - `backoff` : exponential (1s, 2s, 4s)
 - `retry_on` : transient errors (DB timeout, lock contention)
 - `no_retry_on` : guard failures, validation errors, business logic failures
- `dead_letter` : After `max_retries` exhausted, emit `pipeline.dead_letter` with original event, error details, and retry history. Dead letter events are root-scoped.

AGENT_SESSION_FAILURE

- `behavior` : Agent exceeds turn budget or crashes. Session terminated.
- `retry_policy` :
 - `max_retries` : 1
 - `note` : Agent sessions are expensive (LLM calls). Retry once, then dead-letter.

- `timer_failure` :
 - `behavior` : Timer event treated as regular event. Same retry policy as handler failures.
- `chain_depth_limit` :
 - `description` : Maximum number of chained event emissions in a single causal chain. Prevents $A \rightarrow B \rightarrow A \rightarrow B$ infinite loops.
 - `max_depth` : 50
 - `behavior` : Each event carries a `chain_depth` counter (starting at 0). When a handler emits a new event, `chain_depth` increments. If `chain_depth` exceeds `max_depth`, the event is routed to `pipeline.dead_letter` instead of being delivered. The chain is broken.

DEAD_LETTER_SCHEMA

- `description` : Structured payload for `pipeline.dead_letter` events.
- `fields` :
 - `original_event` : string — the event name that failed
 - `original_payload` : object — full payload of the failed event
 - `entity_id` : string — entity the event was targeting
 - `flow_instance` : string — flow instance path
 - `failure_type` : string — `handler_error` | `chain_depth_exceeded` | `retry_exhausted`
 - `error_message` : string — human-readable error description
 - `retry_count` : integer — number of retries attempted before dead-lettering
 - `chain_depth` : integer — current chain depth (for `chain_depth_exceeded`)
 - `handler_node` : string — which system node was handling (if `handler_error`)
 - `timestamp` : string — ISO 8601
- `chain_depth_behavior` :
 - `description` : What happens when `chain_depth` exceeds the limit (default: 50).
 - `intercepted_event` : The event that would exceed the limit is NOT emitted. It is intercepted before entering the event loop. It does NOT appear in `emitted_events`.
 - `dead_letter_record` : A `dead_letters` row is created with `failure_type`: `chain_depth_exceeded`, containing the intercepted event name, payload, and the chain depth at interception.
 - `triggering_handler_outcome` : The handler that attempted to emit the chain-exceeding event gets outcome: success. The handler itself succeeded — it is the EMITTED event that was intercepted. The handler's other side effects (state changes, gate sets, data writes) still commit.

- **note** : Chain depth overflow is NOT a handler failure. It is an emission interception. The `dead_letter` is for the downstream event, not the handler that produced it.

boot_verification

- **description** : The platform runs contract verification at boot, after loading all contracts and before starting any nodes or agents. If any check fails, boot aborts. These checks are the authoritative integration test.

CHECKS

- **id** : `event_chain_integrity`; **severity** : warning
- **id** : `event_consumer_exists`; **severity** : warning
- **id** : `event_producer_exists`; **severity** : warning
- **id** : `payload_field_coverage`; **severity** : error
- **id** : `condition_payload_alignment`; **severity** : error
- **id** : `condition_policy_alignment`; **severity** : warning
- **id** : `state_machine_coherence`; **severity** : error
- **id** : `required_agents_match`; **severity** : error
- **id** : `handler_field_compliance`; **severity** : error
- **id** : `tool_resolution`; **severity** : warning
- **id** : `prompt_exists`; **severity** : warning
- **id** : `produces_drift`; **severity** : warning
- **id** : `invalid_field_detection`; **severity** : error
- **id** : `policy_conflict_detection`; **severity** : warning
- **id** : `event_cycle_detection`; **severity** : error
- **id** : `dialect_compliance`; **severity** : error
- **id** : `single_node_per_event`; **severity** : error
- **id** : `config_from_payload_alignment`; **severity** : error
- **id** : `phantom_produces`; **severity** : warning

- `severity_behavior` :
 - `error` : Boot aborts. System does not start.
 - `warning` : Boot continues. Warning logged with finding details.
- `reference_implementation` : `verify.py` implements structural checks (payload fields, handler fields, state machine, tools, dialect, cycles, produces). It does NOT implement: CEL parsing, pin wiring, permission validation, entity schema coverage, or namespace collision detection. Those require the Go runtime loader.
- `check_count` : 19 checks (10 error, 9 warning)

compliance

Boot-time and runtime compliance enforcement.

runtime_enforcement

- `description` : Checks the platform performs during execution.

REQUIRED_CHECKS

- Tool calls validated against tool schema before execution
- Tool calls checked against agent permissions before execution
- Message delivery checked against agent message scope permission
- State `state_changes` only follow declared `advances_to` paths
- Guards evaluated before state advancement – block if false
- Events validated against payload schema before publish
- Accumulation is idempotent – duplicate events do not double-count
- Permission checks read from `agent.permissions`, not hardcoded policy functions
- Scan mode behavior read from `policy.scan_modes`, not hardcoded
- Manager fallback read from `agent.manager_fallback`, not hardcoded role mapping
- Workspace class read from `agent.workspace_class`, not hardcoded

testing

- `description` : Required test coverage for spec compliance.

REQUIRED_TESTS

- Boot validation passes with current contracts (all checks green)
- Every declared handler has a matching `Handle()` implementation
- Every state `state_change` follows declared `advances_to` paths
- Every emitted event has a payload schema in `events.yaml`
- Every agent subscription resolves to an event some node or agent produces
- Permission enforcement blocks unauthorized tool calls
- Accumulation is idempotent under duplicate event delivery
- Crash recovery replays undelivered events correctly
- Namespace substitution works for multi-instance flows

handler_execution

- `description` : How the runtime executes event handlers.

RULES

- Match inbound event to handler in owning node
- If handler has `guard`: evaluate condition. If false, reject or kill.
- If handler has `accumulate`: track arrival, check completion, proceed only on complete.
- If handler has `advances_to`: update entity state.
- If handler has `emits`: publish follow-up event.
- If handler has `sets_gate`: update entity gate state.
- If handler has `data_accumulation`: write payload fields to entity.
- If handler has `on_complete`: evaluate conditions, execute matching branch.
- If handler has `rules`: match payload field, emit matching event.
- If handler has `action` that is a platform action: call registered platform action.

product_boundary

- `description` : Rules for product-specific code isolation.

RULES

- Product-specific Go code lives only in approved product packages (e.g., `internal/runtime/pipeline/product/`)
- Generic runtime packages contain zero product references
- Platform actions are platform action, not imported by generic code
- CI guard fails if product literals appear in generic non-test Go files

flow_coherence

- Every flow declared in `package.yaml` must have a directory under `flows/`
- Root-level nodes must not duplicate flow-level node IDs
- All handoff events must have both an emitter (source flow) and subscriber (target flow or declarative handoff)

node_coherence

- Every event in `event_handlers` must appear in `subscribes_to`
- Every event in `produces` must have a payload schema in `events.yaml`
- Guards must reference fields that exist on the entity or in `policy.yaml`
- `advances_to` states must be reachable from the current flow state space

agent_coherence

- Every tool in `tools_tier2` must exist in `tools.yaml`
- Every event in `emit_events` must have a payload schema in `events.yaml`
- Agent permissions must include required permissions for their tools
- Every subscription event must be emitted by some node or agent
- `boot_enforcement` : All boot checks are defined in `engine.boot_verification.checks`. This is the single authoritative list.

permissions_model

Agents have a permissions list controlling what capabilities they can exercise. The platform enforces permissions at tool execution time and message routing time. Workflows define permissions per agent in their agent registry. Optional `permission_bundles` in policy files provide shorthand for common sets.

- `enforcement` : Before executing any tool call, the platform checks the calling agent's permissions list. If the tool requires a permission the agent lacks, the call is rejected. Message scope is enforced at `agent_message` delivery per the scoping rules defined in `permission_scoping`.
- `workflow_extensions` : Workflows may define additional permissions beyond the platform set. The platform passes unrecognized permissions to workflow-registered handlers. Example: a product workflow might define `approve_deployment` or `override_scoring`.

permissions

- `agent_fire`
- `agent_hire`
- `agent_reconfigure`
- `approve_spend`
- `configure_routing`
- `create_flow_instance`
- `human_task_decide`
- `human_task_request`
- `mailbox_send`
- `message_flow`
- `message_peers`
- `schedule`

bundle_semantics

- `description` : An agent may declare `permissions_bundle` and/or explicit permissions. If both are present, the explicit permissions list EXTENDS the bundle. Duplicates are deduplicated. The bundle is expanded first, then explicit permissions are added.

permission_scoping

- `description` : Defines the target scope for each permission. The platform enforces these constraints at tool execution time. All scoping is flow-instance-local — no permission grants cross-flow access. Cross-flow communication is via events only.

MESSAGING

- `message_peers` :
 - `scope` : Agents sharing the same `manager_fallback` value, within the same flow instance.
 - `rule` : Caller and target must be in the same flow instance AND `target.manager_fallback == caller.manager_fallback`.
- `message_flow` :
 - `scope` : Any agent in the same flow instance.
 - `rule` : Caller and target must be in the same flow instance. No further constraint.
- `cross_flow` : Not supported. `agent_message` delivery is rejected if target is in a different flow instance. Cross-flow communication uses events (`emit` + `subscribe`).

MANAGEMENT

- `description` : `agent_hire`, `agent_fire`, `agent_reconfigure` target constraints.
- `scope` : Same flow instance, target below caller in `manager_fallback` chain.

rules

- Caller and target must be in the same flow instance.
- Target's `manager_fallback` chain must include the caller. (A can manage B if `B.manager_fallback == A`, or `B.manager_fallback.manager_fallback == A`, etc.)
- An agent cannot manage itself.
- An agent cannot manage its own `manager_fallback` ancestor.

ROUTING

- `description` : `configure_routing` target constraints.
- `scope` : Self + manageable agents (same rules as management scope).

rules

- Caller can modify routes for itself.

- `Caller can modify routes for any agent it can manage (per management scope).`
- `Caller cannot modify routes for agents outside its flow instance.`
- `manager_fallback_semantics` :
 - `description` : `manager_fallback` is an escalation path, not a messaging or management grant. It determines where dead-lettered agent sessions route to and defines the management hierarchy. It does NOT grant the fallback target messaging access to the agent, and it does NOT allow messaging across flow boundaries even when the fallback target is in a different flow.
 - `cross_flow_fallback` : When `manager_fallback` references an agent in a parent flow (e.g., an operating agent falls back to a root-level coordinator), escalation produces an event (`pipeline.dead_letter` or equivalent), not a direct message. The platform routes the escalation event through the normal event loop.

tool_model

The platform owns ALL tool schema serving. It reads tool definition YAML, generates MCP tool definitions, validates calls against schemas, and routes to handlers. Tool handlers are either platform-builtin or workflow-registered.

platform_builtin_tools

- `description` : Authoritative list of platform-provided tools. Defined here only.
- `tools` :
 - `create_flow_instance` : Create a new dynamic flow instance from a template. Handler action only.
 - `record_evidence` : Append payload to entity accumulator. Requires `evidence_target` field on the handler specifying which accumulator key to write to.
 - `create_entity` : Create a new entity row in `entity_state`.
 - `query_entities` : Query entities by state, fields, or aggregation.
 - `agent_message` : Send a message to another agent (auto-granted to all agents).
 - `mailbox_send` : Send an item to the human mailbox (auto-granted to all agents).
- `note` : `create_flow_instance` is a handler ACTION field value, not a tool agents can call. It is listed here for completeness but invoked via `action: create_flow_instance` in node handlers.

- `handler_registration` : At bootstrap, the workflow module registers handlers for its tools. The platform provides the MCP gateway and schema validation. One tools file per workflow containing all tool schemas (universal + workflow-specific). The platform does not need separate tool files.

custom_tool_schema

- `description` : Accepted fields for tool definitions in tools.yaml.
- `required_fields` :
 - `description` : string — human-readable description of what the tool does
- `optional_fields` :
 - `handler_type` : string — workflow_registered (default), api_call, builtin
 - `input_schema` : object — parameter names and types the tool accepts
 - `output_schema` : object — field names and types the tool returns
 - `parameters` : object — alias for input_schema (accepted by loader)
 - `returns` : object — alias for output_schema (accepted by loader)
- `not_accepted` :
 - `endpoint` : NOT a valid field. API endpoint configuration belongs in agent config or environment, not tool schema.
 - `type` : NOT a valid field. Use handler_type instead.

prompt_templating

Agent prompts are markdown files in the prompts/ directory. They may contain `{{variable}}` placeholders that the platform substitutes from policy.yaml at agent session creation time. The substitution is simple string replacement — no logic, no conditionals. Variables not found in policy.yaml are left as-is (fail-open for forward compatibility).

convention

- `prompt_path` : prompts/{agent-id}.md
- `variable_syntax` : `{{variable_name}}`
- `variable_source` : policy.yaml

variable_resolution

- **description** : Prompt variables ({{token}}) are resolved from multiple sources in priority order. All sources must be scanned by compliance tests.

RESOLUTION_ORDER

- **source** : instance_variables (schema.yaml); **description** : Variables declared in the flow schema. Populated at flow instance creation via config_from.; **example** : vertical_name, mandate_document, geography
- **source** : policy (policy.yaml); **description** : Policy values from flow and root policy files.; **example** : monthly_api_cap, growth_budget, max_escalations
- **source** : entity_state fields; **description** : Entity fields written by data_accumulation. Available at render time from entity context.; **example** : composite_score, business_brief
- **source** : runtime tokens; **description** : Platform-injected tokens not declared in contracts. Hardcoded allowlist.; **example** : current_date, agent_id, flow_instance_path
- **compliance_test_sources** : TestPromptVariablesComplete must scan ALL of these for each prompt directory: schema.yaml (instance_variables.variables), policy.yaml (all keys), agents.yaml (prompt_inputs), and the runtime-token allowlist. A variable is valid if it appears in ANY of these sources.

flow_model

A Flow is the universal building block. Every level of the system is a flow — from the root flow down to the smallest sub-flow. A flow can contain child flows. The root flow is what others call “the root flow.” There is no separate project concept (removed).

flow_package

- **description** : A flow is a directory with a standard file set. package.yaml is the manifest — it declares identity, child flows, and handoffs. All other files are optional depending on what the flow does.
- **files** :
 - **package.yaml** : Flow manifest (required)
 - **schema.yaml** : Pins, states, required_agents (required)

- `agents.yaml` : Agent definitions (required if flow has agents)
- `events.yaml` : Event schemas (optional — flow may only compose children)
- `nodes.yaml` : System nodes with handlers (optional)
- `tools.yaml` : Flow-specific tools (optional — inherits root)
- `policy.yaml` : Flow-specific config (optional — inherits root)
- `prompts/` : Agent prompt files (optional)
- `flows/` : Child flows (optional)

nesting

- `description` : Flows nest recursively. A flow's `package.yaml` declares child flows in its `flows:` list. Each child is a directory under `flows/` with the same structure. There is no depth limit. The platform discovers flows by walking the tree from the root.
- `discovery_algorithm` : 1. Read `package.yaml` in the current directory. 2. Register this flow (its nodes, events, agents, schema, etc.). 3. For each entry in `flows:` list, resolve the child directory under `flows/`. 4. Recurse: read the child's `package.yaml` and repeat from step 1. 5. Stop when a flow has no `flows:` list or the directory doesn't exist.
- `root_flow` : The root flow is the entry point. It has flow-level agents, nodes, events, policy. It declares the top-level child flows. It is structurally identical to any other flow.

example

```
product/                                # root flow
  package.yaml                          # name: product, flows: [discovery, scoring, ...]
  nodes.yaml                           # portfolio-node
  events.yaml                          # cross-flow events
  agents.yaml                          # 3 root-level agents
  schema.yaml                          # root flow pins
  flows/
    discovery/                          # child flow
      package.yaml                      # name: discovery, flows: [] (leaf)
      nodes.yaml
      events.yaml
      schema.yaml
      agents.yaml
      prompts/
    validation/                         # child flow with sub-flows
      package.yaml                      # name: validation, flows: [research, speccing, ...]
      nodes.yaml                       # gate orchestrator
```

```

schema.yaml
flows/
  research/                # grandchild flow
    package.yaml
    nodes.yaml
    agents.yaml
    prompts/
    speccing/
    ...

```

pins

- **description** : Flows have typed input/output pins, like IC chips in a circuit. The platform validates pin wiring at boot.
- **input_event_pins** :
 - **description** : Events the flow subscribes to. Required pins must be wired (some other flow emits them) or boot warns.
 - **required** : true
- **output_event_pins** :
 - **description** : Events the flow emits. Optional — subscribers consume them if present, otherwise events are logged only.
 - **required** : false
- **input_data_pins** :
 - **description** : Entity fields the flow reads. Required pins must be written by a prior flow or initial data.
 - **required_configurable** : true
- **output_data_pins** :
 - **description** : Entity fields the flow writes. Always written. No two flows may write the same field.
 - **conflict_rule** : Two flows writing the same field is a boot error (short circuit).
- **internal_events_convention** : Schema pins declare the flow's EXTERNAL interface only. Internal agent-to-agent events within a flow (e.g., spec.requested between two agents in the same flow) exist in events.yaml but are NOT listed in schema pins. They are implementation details, not part of the flow's public contract. Only events that cross flow boundaries or trigger/complete the flow appear in pins.

required_agents

- **description** : Each flow schema declares `required_agents` — agent role sockets that the root flow must fill. Each entry specifies: role name, events the agent subscribes to, events it emits, and a description. The platform validates at boot that every required role has a matching agent in `agents.yaml`.
- **schema_field** : `required_agents`
- **entry_format** :
 - **role** : Name of the required role
 - **subscribes_to** : Events the agent must subscribe to
 - **emits** : Events the agent must emit
 - **description** : What the agent does in this flow
- **fulfillment** : An agent fulfills a role if its subscriptions include the required `subscribes_to` events (after namespace substitution) and its `emit_events` include the required emits. The flow author maps agents to roles — the platform enforces the contract.

schema_merging

- **description** : When a root flow installs multiple flows, the platform merges their schemas. All output data pins become entity table columns. Input data pins are validated against available outputs. Two flows writing the same field is a conflict (boot error).

cross_flow_events

- **rule** : An event schema is defined ONCE, in the flow that emits it. Consuming flows reference it by absolute path — they do NOT redefine the schema. If both flows need the schema for validation, the consuming flow imports it by reference, not by copy.
- **enforcement** : At boot, if the same event name appears in multiple flow event files, the platform logs a warning. The emitter flow's definition is authoritative.

stateless_flows

- **description** : Some flows are stateless producers — they process events without tracking entity lifecycle. Discovery is an example: it scans, filters, and emits verticals without maintaining per-entity state.

- **schema_for_stateless** : A stateless flow still has schema.yaml but may set initial_state: null and states: []. Events pass through system nodes which accumulate, filter, and emit — but no entity state machine is maintained. The accumulator state is flow-scoped, not entity-scoped.
- **when_to_use** : Producer pipelines, ETL flows, signal processing, fan-out dispatchers.
- **entity_behavior** : In a stateless flow (initial_state: null, states: []), the platform does not create entity_state rows. Events pass through system nodes for accumulation and fan_out but no state machine is maintained. advances_to is invalid in a stateless flow handler (boot error if used). Guards may reference payload and policy but not entity (no entity exists). Accumulators are flow-scoped, keyed by a handler-declared key, not by entity_id.

state_composition

- **description** : How flow-level state machines compose into the entity lifecycle.
- **per_flow_states** : Each flow defines its own state space. An entity in the scoring flow has scoring states. The same entity in the validation flow has validation states. These are SEPARATE state machines — not one unified enum.
- **cross_flow_transitions** : When an entity moves from one flow to another (scoring → validation), the transition is: 1. Scoring flow handler emits a cross-flow event (e.g., vertical.shortlisted). 2. Validation flow node handles it, advances entity to validation initial state (researching). 3. The entity now has a validation flow state. Its scoring flow state is terminal (shortlisted). There is no “global state” — each flow tracks its own state for the entity.
- **killed_across_flows** : killed is a terminal state in every flow. When validation kills an entity, it emits vertical.killed + vertical.killed_backprop. The scoring flow handles killed_backprop and advances the entity to killed in the scoring state machine too.

versioning

Platform and products version independently.

platform_versioning

- **scheme** : semver (major.minor.patch)
- **major** : Breaking changes to flow model, handler fields, boot validation, or contract formats
- **minor** : New execution primitives, new handler fields (backward compatible), new compliance checks
- **patch** : Bug fixes, documentation, clarifications

product_versioning

- **scheme** : semver (major.minor.patch)
- **major** : Flow restructuring, agent hierarchy changes, breaking event schema changes
- **minor** : New agents, new events, new tools, threshold adjustments
- **patch** : Prompt improvements, policy tuning, bug fixes

compatibility

- **declaration** : Products declare platform_version in package.yaml using semver range (e.g., `>=1.0.0`)
- **enforcement** : Platform checks product platform_version at boot. Rejects if platform version is outside range.
- **forward_compatible** : Platform minor/patch releases do not break existing products
- **example** : Product v3.0.0 declares platform_version: “`>=1.0.0`”. Works on platform 1.0.0, 1.1.0, 1.9.0. May not work on 2.0.0.

platform_tables

Platform-owned infrastructure tables. These exist regardless of which product contracts are loaded. They are NOT entity tables (those are contract-driven from entity_schema). The platform generates DDL for these at boot via `GeneratePlatformTableDDLs`.

tables

EVENTS

- **description** : Append-only event store. Every event persisted before delivery. Three scopes: entity (entity_id + flow_instance set), flow (flow_instance set, entity_id NULL), global (both NULL — system bootstrap, runtime recovery, admin commands). idempotency_key for external dedup. source_event_id for causal chain tracing.

DDL

```

CREATE TABLE events (
  event_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_name        TEXT NOT NULL,
  entity_id         UUID,
  flow_instance     TEXT,
  scope            TEXT NOT NULL DEFAULT 'entity' CHECK (scope IN ('entity', 'flow', 'global')),
  payload           JSONB NOT NULL DEFAULT '{}',
  chain_depth       INTEGER NOT NULL DEFAULT 0,
  produced_by       TEXT,
  produced_by_type  TEXT CHECK (produced_by_type IN ('node', 'agent', 'platform', 'external')),
  handler_node      TEXT,
  idempotency_key   TEXT,
  source_event_id   UUID,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  INDEX idx_events_entity (entity_id, created_at) WHERE entity_id IS NOT NULL,
  INDEX idx_events_flow (flow_instance, event_name) WHERE flow_instance IS NOT NULL,
  INDEX idx_events_global (event_name, created_at) WHERE scope = 'global',
  INDEX idx_events_idempotency (idempotency_key) WHERE idempotency_key IS NOT NULL,
  INDEX idx_events_source (source_event_id) WHERE source_event_id IS NOT NULL
);

```

EVENT_DELIVERIES

- **description**: Tracks delivery of events to subscribers (nodes and agents). One row per event × subscriber.

DDL

```

CREATE TABLE event_deliveries (
  delivery_id       UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id          UUID NOT NULL REFERENCES events(event_id),
  subscriber_type   TEXT NOT NULL CHECK (subscriber_type IN ('node', 'agent')),
  subscriber_id     TEXT NOT NULL,
  status            TEXT NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'delivered', 'failed', 'dead_letter')),
  retry_count       INTEGER NOT NULL DEFAULT 0,
  last_error        TEXT,
  delivered_at      TIMESTAMPTZ,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  INDEX idx_deliveries_status (status, created_at),
  INDEX idx_deliveries_event (event_id)
);

```

EVENT_RECEIPTS

- **description** : Handler completion acknowledgement. One receipt per event × subscriber. Replaces both pipeline_receipts and system_node_ledger. entity_id nullable for global events. subscriber_type + subscriber_id identify the handler generically (node, agent, or platform subsystem). idempotency_key used by system_node_runner for replay dedup — if a receipt exists for this key, the handler skips re-execution. duration_ms tracks handler execution time for observability. side_effects records emitted events and state changes for flight recorder.

DDL

```
CREATE TABLE event_receipts (
  receipt_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id        UUID NOT NULL REFERENCES events(event_id),
  subscriber_type TEXT NOT NULL CHECK (subscriber_type IN ('node', 'agent', 'platform')),
  subscriber_id   TEXT NOT NULL,
  entity_id       UUID,
  flow_instance   TEXT,
  outcome         TEXT NOT NULL CHECK (outcome IN ('success', 'reject', 'discard', 'kill',
    'escalate', 'dead_letter', 'no_op')),
  state_before    TEXT,
  state_after     TEXT,
  side_effects     JSONB NOT NULL DEFAULT '{}',
  duration_ms     INTEGER,
  idempotency_key TEXT,
  processed_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (event_id, subscriber_id),
  INDEX idx_receipts_entity (entity_id) WHERE entity_id IS NOT NULL,
  INDEX idx_receipts_subscriber (subscriber_id, processed_at),
  INDEX idx_receipts_idempotency (idempotency_key) WHERE idempotency_key IS NOT NULL
);
```

ENTITY_STATE

- **description** : Current state of every entity. This IS the entity registry — no separate registry table. slug and name are top-level columns for fast lookup/display without JSONB queries. entity_type distinguishes entity kinds within a flow (e.g., vertical, opco, ticket). fields JSONB holds all contract-driven entity fields. Generic store reads use entity_state for listing, filtering, and lookup.

DDL

```
CREATE TABLE entity_state (
  entity_id      UUID PRIMARY KEY,
  flow_instance   TEXT NOT NULL,
  entity_type     TEXT NOT NULL DEFAULT 'default',
```

```

slug          TEXT,
name          TEXT,
current_state TEXT NOT NULL,
gates         JSONB NOT NULL DEFAULT '{}',
fields        JSONB NOT NULL DEFAULT '{}',
accumulator   JSONB NOT NULL DEFAULT '{}',
revision      INTEGER NOT NULL DEFAULT 0,
entered_state_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
INDEX idx_entity_flow (flow_instance, current_state),
INDEX idx_entity_state (current_state),
INDEX idx_entity_type (entity_type),
INDEX idx_entity_slug (slug) WHERE slug IS NOT NULL
);

```

AGENTS

- **description** : Runtime agent registry. One row per agent instance. Carries the full runtime descriptor: role (from agents.yaml), parent_agent_id (managerial hierarchy), config (agent-specific configuration from contracts), subscriptions/emit_events/tools/permissions (materialized from contracts at boot for fast runtime lookup). flow_instance nullable for root-level agents not scoped to any flow. entity_id nullable — set only for agents scoped to a specific entity (rare). The contract in agents.yaml is the source of truth. This table is the runtime materialization. llm_backend specifies the LLM invocation backend — api (production), cli_test (test harness), mock (fixture replay), local (local model). This is a deployment/agent property, not a session property. agent_sessions.runtime_mode tracks conversation scope only.

DDL

```

CREATE TABLE agents (
  agent_id      TEXT PRIMARY KEY,
  flow_instance TEXT,
  role         TEXT NOT NULL,
  model_tier   TEXT NOT NULL,
  llm_backend   TEXT NOT NULL DEFAULT 'api' CHECK (llm_backend IN ('api', 'cli_test',
    'mock', 'local')),
  conversation_mode TEXT NOT NULL CHECK (conversation_mode IN ('task', 'session',
    'session_per_entity')),
  parent_agent_id TEXT,
  entity_id      UUID,
  config         JSONB NOT NULL DEFAULT '{}',
  subscriptions  JSONB NOT NULL DEFAULT '[]',
  emit_events    JSONB NOT NULL DEFAULT '[]',
  tools         JSONB NOT NULL DEFAULT '[]',
  permissions    JSONB NOT NULL DEFAULT '[]',

```

```

status          TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'paused',
                    'terminated')),
turn_count       INTEGER NOT NULL DEFAULT 0,
last_active_at   TIMESTAMPTZ,
created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
INDEX idx_agents_flow (flow_instance) WHERE flow_instance IS NOT NULL,
INDEX idx_agents_role (role),
INDEX idx_agents_status (status),
INDEX idx_agents_parent (parent_agent_id) WHERE parent_agent_id IS NOT NULL
);

```

AGENT_SESSIONS

- **description**: Agent conversation sessions. Supports three scopes: entity (one session per agent × entity), flow (one session per agent × flow instance), global (one session per agent, shared across all entities). `scope_key` is the lookup key used by the runtime: for entity scope it is the `entity_id`, for flow scope it is the `flow_instance` path, for global scope it is the literal “global”. `runtime_mode` matches the agent conversation_mode from contracts. `lease_holder/lease_expires_at` for distributed session locking.

DDL

```

CREATE TABLE agent_sessions (
    session_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    agent_id         TEXT NOT NULL REFERENCES agents(agent_id),
    entity_id        UUID,
    flow_instance    TEXT,
    scope_key        TEXT NOT NULL,
    scope           TEXT NOT NULL DEFAULT 'entity' CHECK (scope IN ('entity', 'flow', 'global')),
    conversation     JSONB NOT NULL DEFAULT '[]',
    turn_count       INTEGER NOT NULL DEFAULT 0,
    runtime_mode     TEXT NOT NULL DEFAULT 'task' CHECK (runtime_mode IN ('task', 'session',
                    'session_per_entity')),
    runtime_state    JSONB NOT NULL DEFAULT '{}',
    lease_holder     TEXT,
    lease_expires_at TIMESTAMPTZ,
    status          TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'suspended',
                    'terminated')),
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (agent_id, scope_key),
    INDEX idx_sessions_agent (agent_id, status),
    INDEX idx_sessions_entity (entity_id) WHERE entity_id IS NOT NULL,
    INDEX idx_sessions_scope (scope, status),
    INDEX idx_sessions_lease (lease_expires_at) WHERE lease_holder IS NOT NULL
);

```

scope_rules

- **entity** : entity_id NOT NULL, flow_instance NOT NULL, scope_key = entity_id. One session per agent per entity.
- **flow** : entity_id NULL, flow_instance NOT NULL, scope_key = flow_instance. One session per agent per flow instance.
- **global** : entity_id NULL, flow_instance NULL, scope_key = “global”. One session per agent across the entire system.
- **lookup_pattern** : Runtime acquires sessions via (agent_id, scope_key). The scope_key is derived from runtime_mode: task → no session (stateless), session_per_entity → scope_key = entity_id, session → scope_key = flow_instance or “global” depending on agent declaration.

ROUTING_RULES

- **description** : Unified routing table for both contract-defined patterns and materialized instance routes. Replaces the need for a separate flow_instance_routes table. At boot: contract subscriptions inserted as pattern rows (is_wildcard may be true). On create_flow_instance: wildcard patterns expanded into concrete rows (is_materialized = true, materialized_from = parent rule_id). On flow instance termination: materialized rows set to inactive. Event dispatch: match concrete routes first (is_materialized = true, exact match), fall back to wildcard patterns.

DDL

```
CREATE TABLE routing_rules (
  rule_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_pattern    TEXT NOT NULL,
  subscriber_type  TEXT NOT NULL CHECK (subscriber_type IN ('node', 'agent')),
  subscriber_id    TEXT NOT NULL,
  flow_instance    TEXT,
  source_flow      TEXT,
  is_wildcard      BOOLEAN NOT NULL DEFAULT FALSE,
  is_materialized  BOOLEAN NOT NULL DEFAULT FALSE,
  materialized_from UUID,
  status           TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'inactive')),
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  INDEX idx_routing_event (event_pattern, status),
  INDEX idx_routing_subscriber (subscriber_id),
  INDEX idx_routing_flow (flow_instance) WHERE flow_instance IS NOT NULL,
  INDEX idx_routing_materialized (materialized_from) WHERE materialized_from IS NOT NULL
);
```

lifecycle

- **boot** : Load contract subscriptions → INSERT pattern rows. Clear stale materialized rows from previous boot.
- **create_flow_instance** : For each wildcard pattern matching the new instance path → INSERT materialized row with concrete flow_instance.
- **terminate_flow_instance** : UPDATE materialized rows for this flow_instance → status = inactive.
- **event_dispatch** : SELECT WHERE event_pattern matches AND status = active. Concrete rows (is_materialized) take priority over wildcards.

MAILBOX

- **description** : Human-in-the-loop task queue. Payload-centric model — all task-specific data in payload JSONB. item_type distinguishes task kinds: human_task, review_request, approval, alert, operational_decision. human_tasks are represented as mailbox items with item_type = “human_task”. from_agent identifies the requesting agent. summary provides a human-readable description. decision_notes captures the human rationale. notified tracks whether the human was alerted. source_event_id links back to the triggering event for traceability.

DDL

```
CREATE TABLE mailbox (
  item_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  entity_id        UUID,
  flow_instance    TEXT,
  scope            TEXT NOT NULL DEFAULT 'entity' CHECK (scope IN ('entity', 'flow', 'global')),
  item_type        TEXT NOT NULL,
  source_event_id  UUID,
  from_agent       TEXT,
  severity         TEXT NOT NULL DEFAULT 'normal' CHECK (severity IN ('normal', 'urgent',
    'critical')),
  summary          TEXT,
  payload          JSONB NOT NULL DEFAULT '{}',
  status           TEXT NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'decided',
    'expired', 'cancelled')),
  decision         TEXT,
  decision_notes   TEXT,
  decided_by       TEXT,
  decided_at       TIMESTAMPTZ,
  notified         BOOLEAN NOT NULL DEFAULT FALSE,
  expires_at       TIMESTAMPTZ,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  INDEX idx_mailbox_status (status, severity, created_at),
  INDEX idx_mailbox_entity (entity_id) WHERE entity_id IS NOT NULL,
  INDEX idx_mailbox_type (item_type, status),
```

```
INDEX idx_mailbox_global (scope, status) WHERE scope = 'global'
);
```

SPEND_LEDGER

- **description**: LLM token and API cost tracking. One row per agent invocation.

DDL

```
CREATE TABLE spend_ledger (
  ledger_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  entity_id      UUID,
  flow_instance  TEXT NOT NULL,
  agent_id       TEXT NOT NULL,
  model          TEXT NOT NULL,
  input_tokens   INTEGER NOT NULL DEFAULT 0,
  output_tokens  INTEGER NOT NULL DEFAULT 0,
  cost_usd       NUMERIC(10,6) NOT NULL DEFAULT 0,
  invocation_type TEXT NOT NULL,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  INDEX idx_spend_flow (flow_instance, created_at),
  INDEX idx_spend_agent (agent_id, created_at)
);
```

FLOW_INSTANCES

- **description**: Registry of all flow instances (static and dynamic). One row per instance.

DDL

```
CREATE TABLE flow_instances (
  instance_id    TEXT PRIMARY KEY,
  flow_template  TEXT NOT NULL,
  mode           TEXT NOT NULL CHECK (mode IN ('static', 'template')),
  parent_instance TEXT,
  config         JSONB NOT NULL DEFAULT '{}',
  status         TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'draining',
    'terminated')),
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  terminated_at  TIMESTAMPTZ
);
```

TIMERS

- **description** : Durable timer and scheduled task store. Persisted across restarts. Entity-scoped timers: `entity_id` and `flow_instance` set, tied to a specific entity lifecycle. Global timers: `entity_id` and `flow_instance` NULL, used for recurring platform-level work (periodic scans, health checks, digest compilation). `fire_payload` carries context when the timer fires. `owner_node` OR `owner_agent` identifies who declared the timer (mutually exclusive). `recurrence_cron` supports cron expressions for global recurring schedules. `recurrence_interval` supports duration strings (e.g., “24h”) for entity-scoped recurring timers. `task_type`: timer (one-shot entity), `scheduled_task` (recurring entity), `deadline` (hard cutoff), `global_recurring` (platform-level periodic work).

DDL

```
CREATE TABLE timers (
  timer_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  timer_name        TEXT NOT NULL,
  entity_id         UUID,
  flow_instance     TEXT,
  fire_event        TEXT NOT NULL,
  fire_payload      JSONB NOT NULL DEFAULT '{}',
  fire_at           TIMESTAMPTZ NOT NULL,
  recurring         BOOLEAN NOT NULL DEFAULT FALSE,
  recurrence_cron   TEXT,
  recurrence_interval TEXT,
  owner_node        TEXT,
  owner_agent       TEXT,
  task_type         TEXT NOT NULL DEFAULT 'timer' CHECK (task_type IN ('timer',
    'scheduled_task', 'deadline', 'global_recurring')),
  status            TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'fired',
    'cancelled', 'expired')),
  fired_at          TIMESTAMPTZ,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  INDEX idx_timers_fire (status, fire_at),
  INDEX idx_timers_entity (entity_id) WHERE entity_id IS NOT NULL,
  INDEX idx_timers_flow (flow_instance) WHERE flow_instance IS NOT NULL,
  INDEX idx_timers_global (task_type, status) WHERE entity_id IS NULL
);
```

scope_rules

- **entity_scoped** : `entity_id` NOT NULL, `flow_instance` NOT NULL. Tied to entity lifecycle. Cancelled when entity reaches terminal state.
- **global** : `entity_id` NULL, `flow_instance` NULL. Platform-level. Not tied to any entity. Survives entity lifecycle.

- **flow_scoped** : entity_id NULL, flow_instance NOT NULL. Tied to flow instance. Cancelled when flow instance terminates.

DEAD_LETTERS

- **description** : Failed events after retry exhaustion or chain depth overflow.

DDL

```
CREATE TABLE dead_letters (
  dead_letter_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  original_event_id UUID REFERENCES events(event_id),
  original_event    TEXT NOT NULL,
  original_payload  JSONB NOT NULL DEFAULT '{}',
  entity_id         UUID,
  flow_instance     TEXT NOT NULL,
  failure_type      TEXT NOT NULL CHECK (failure_type IN ('handler_error',
    'chain_depth_exceeded', 'retry_exhausted')),
  error_message     TEXT,
  retry_count       INTEGER NOT NULL DEFAULT 0,
  chain_depth       INTEGER NOT NULL DEFAULT 0,
  handler_node      TEXT,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  INDEX idx_dead_letters_flow (flow_instance, created_at),
  INDEX idx_dead_letters_type (failure_type)
);
```

SCHEMA_VERSION

- **description** : Platform schema versioning. Tracks which version of the platform DDL has been applied. The platform checks this at boot: if the stored version < current platform version, it applies migrations. Products never read or write this table.

DDL

```
CREATE TABLE schema_version (
  id                INTEGER PRIMARY KEY DEFAULT 1 CHECK (id = 1),
  platform_version  TEXT NOT NULL,
  applied_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  migration_log     JSONB NOT NULL DEFAULT '[]'
);
```

notes

- `product_tables` : Product-specific tables (entity fields, custom state) are generated from `entity_schema` in `package.yaml`. Those are contract-driven, not listed here.
- `migrations` : Platform table schema is versioned with the platform version. The platform generates DDL at boot and applies migrations if the schema version changed. Products never modify platform tables directly.
- `index_syntax` : INDEX declarations are logical — the implementer may use CREATE INDEX statements or database-specific syntax. The spec declares intent, not SQL dialect.

diagnostics_encoding

- `description` : Runtime diagnostics are encoded into existing platform tables. No dedicated diagnostics table.

RUNTIME_LOG_ENCODING

- `table` : events
- `event_name` : `platform.runtime_log`
- `scope` : global (`entity_id` NULL, `flow_instance` NULL)
- `produced_by_type` : platform
- `payload_fields` :
 - `log_level` : string (debug, info, warn, error, fatal)
 - `component` : string (boot, event_loop, timer_engine, agent_manager, etc.)
 - `message` : string
 - `details` : object (structured context)
 - `stack_trace` : string (for errors only)
- `note` : Queryable via: `SELECT * FROM events WHERE event_name = "platform.runtime_log" AND payload->>log_level = "error"`

PIPELINE_TRANSITION_ENCODING

- `table` : event_receipts
- `mapping` :
 - `state_before` : `event_receipts.state_before`

- `state_after` : `event_receipts.state_after`
- `events_emitted` : `event_receipts.side_effects.emitted_events` (JSONB array)
- `drop_reason` : `event_receipts.outcome` (reject/discard/kill) + `side_effects.drop_reason`
- `duration` : `event_receipts.duration_ms`
- **note** : Every handler execution already produces an `event_receipt`. The receipt captures `state_before`, `state_after`, `outcome`, `duration`, and `side_effects`. This IS the pipeline transition record. No separate table needed.
- **entity_registry_policy** : There is no separate entity registry table. `entity_state` IS the registry. Generic reads (list entities, find by slug, count by state) query `entity_state` directly. Contract-driven entity tables are optional optimizations — they materialize fields JSONB into typed columns for query performance. Not required for correctness.

entity_metrics_convention

- **description** : Metrics are entity fields in `entity_state.fields` JSONB. No separate table.
- **canonical_fields** :
 - `name` : `entity_state.name` (top-level column)
 - `stage` : `entity_state.current_state` (top-level column)
 - `users_total` : `entity_state.fields->'users_total'` (JSONB)
 - `mrr` : `entity_state.fields->'mrr'` (JSONB, numeric)
 - `spend` : Aggregate from `spend_ledger`: `SELECT SUM(cost_usd) FROM spend_ledger WHERE entity_id = X`
- **digest_queries** : Digest/reporting code should query `entity_state` for current metrics and `spend_ledger` for cost aggregates. No `entity_metrics` table.
- **migration_policy** : `schema_version` is a platform table. It survives the legacy SQL deletion. `MigrationSpec` and `ApplyManagedMigrations` remain as platform APIs — they apply DDL diffs between platform versions. `ApplyMigrationFile` is DELETED (no more file-based migrations). The boot sequence: read `schema_version` → compare to current platform version → generate DDL diff → apply → update `schema_version`.

test_bootstrap

- **description** : After legacy SQL deletion, test bootstrap creates tables from two sources only.

- **sources** :
 - **platform_tables** : All tables from `platform_tables.tables` in `platform-spec.yaml`. Always created.
 - **entity_tables** : Generated from `entity_schema` in the flow contracts loaded for the test. Optional — only if the test uses entity fields beyond the JSONB.
- **generic_store_tests** : Generic store tests (`postgres_smoke_test.go`, etc.) bootstrap using `platform_tables` only. They create entities in `entity_state`, events in `events`, sessions in `agent_sessions`. No product contracts needed. No legacy SQL files. No migrations/ directory.
- **product_integration_tests** : Tests that need Empire-specific entity tables load Empire contracts and generate DDL from `entity_schema`. These tests are product tests, not platform tests.
- **bootstrap_function** : `BootstrapPlatformTables(spec PlatformSpec)` — reads `platform_tables.tables`, generates DDL, executes. Called by both production boot and test setup. Single code path.

credentials_policy

- **description** : Webhook secrets, API keys, and other credentials are NOT stored in platform tables. The platform provides a config surface; products choose their storage strategy.
- **options** :
 - **flow_instance_config** : For per-instance secrets (webhook signing keys, API tokens): store in `flow_instances.config` JSONB under a “secrets” key. The platform does not encrypt this field — the product layer or infrastructure (encrypted-at-rest DB, vault sidecar) handles encryption.
 - **external_vault** : For production deployments: use an external secrets manager (HashiCorp Vault, AWS Secrets Manager, etc.). `flow_instances.config` stores a vault reference, not the secret itself.
- **migration_from_legacy** : Legacy code that read `workflow_instances.metadata.credentials` must be rewritten to read from `flow_instances.config.secrets`. This is a product-level migration — `inbound.go` is product code, not platform code. Move it out of generic runtime.
- **platform_guarantee** : The platform never reads, writes, logs, or indexes credential fields. It treats `flow_instances.config` as opaque JSONB. Products own their secrets.

observation_model

Defines exactly what is observable from a test, audit, or flight recorder perspective. This is the external contract of the engine — what an observer sees after a handler runs.

emitted_events

- `description` : The list of events produced by system node handler execution.

INCLUDES

- Events from handler emits field
- Events from handler rules emits
- Events from handler on_complete emits
- Events from handler fan_out emit_per_item
- Events from handler action side effects (e.g., auto_emit_on_create)

EXCLUDES

- Agent fixture emissions (agents are independent, their emissions route through the event loop separately)
- Dead-lettered events (intercepted before delivery, recorded in dead_letters table instead)
- Child flow auto_emit events (visible within the child, not in parent emission log)
- `naming` : Events are recorded with their fully-qualified URI in the parent flow context. When a child flow node emits order.completed, the parent sees child-flow-id/order.completed. Within the same flow, events use local names (no prefix).

PAYLOAD_RULES

- `description` : Rules for constructing emitted event payloads.

construction_order

- 1. Entity fields (base layer – contract-visible fields from entity_state)
- 2. Trigger payload (overlays entity fields – trigger wins on collision)
- 3. Platform fields forced: entity_id, trigger_event_type, current_state
- 4. payload_transform replaces steps 1-2 if present
- `collision_priority` : platform fields > payload_transform > trigger payload > entity fields
- `note` : This means emitted events carry context from both entity state AND the triggering event. If the trigger payload has a field with the same name as an entity field, the trigger value is used. Platform fields (entity_id, trigger_event_type, current_state) are always present and cannot be overridden.

entity_state

- **description** : The entity document after handler execution commits.
- **observable_fields** :
 - **current_state** : The state after `advances_to` (or unchanged if no `advances_to`)
 - **gates** : Map of gate names to boolean values, after `sets_gate` and `clear_gates`
 - **fields** : JSONB of entity fields, after `data_accumulation` writes
 - **accumulator** : JSONB of accumulated items, after `accumulate` step
 - **revision** : Incremented on every handler commit

handler_outcome

- **description** : The result of a single handler execution.
- **values** :
 - **success** : Handler ran to completion. All side effects committed.
 - **reject** : Guard failed with `on_fail`: `reject`. No side effects. Entity unchanged.
 - **discard** : Guard failed with `on_fail`: `discard`. No side effects. No record.
 - **kill** : Guard failed with `on_fail`: `kill`. Entity advanced to terminal state.
 - **escalate** : Guard failed with `on_fail`: `escalate:{event}`. Escalation event emitted instead.
 - **dead_letter** : Handler failed after retry exhaustion OR chain depth exceeded. Event moved to `dead_letters`.
 - **terminal_reject** : Event targeted an entity in terminal state. Rejected before handler execution.
- **agent_vs_node_observability** : System node handler emissions are recorded in the entity emission chain and appear in `emitted_events`. Agent emissions are NOT part of the entity emission chain — they are independent events that enter the event loop separately. An agent emitting `ticket.classified` creates a new event in the events table that is then delivered to subscribers. The agent emission does not appear in the triggering handler's `emitted_events` list.

workspace_model

Defines the filesystem contract for agent sessions. The platform provides three standard mount points and a scoping model. Products define workspace classes that map agent roles to specific scope configurations. If a product declares a `workspace_class` on an agent, the platform **MUST** provide all mounts at the declared scopes.

standard_mounts

- `description` : Three mount points are available to every agent session. Products cannot add new mount points — they configure access and scope for these three via workspace class definitions.

/WORKSPACE

- `description` : Writable working directory for agent output — drafts, artifacts, intermediate files.
- `access` : read-write

scope_options

- `per-agent` – private to one agent, invisible to others
- `per-flow-instance` – shared by all agents in the same flow instance, isolated across instances
- `lifecycle` :
 - `per-agent` : Created at agent registration. Cleaned on agent termination. Persists across sessions for session/session_per_entity conversation modes. Ephemeral for task mode.
 - `per-flow-instance` : Created on `create_flow_instance`. Preserved on instance termination (for post-mortem). Archival is an operational concern.

/DATA

- `description` : Read-only shared reference data. Products populate this via deployment configuration. The platform treats `/data` as opaque — it never reads, writes, indexes, or validates `/data` contents.
- `access` : read-only
- `scope` : global — identical content visible to every agent regardless of workspace class
- `lifecycle` : Mounted at boot. Contents managed outside the platform (deployment, CI, admin).

/OPT/MAS/CONTRACTS

- `description` : Loaded flow contracts. Agents can read their own flow's contract files for self-reference.
- `access` : read-only
- `scope` : global
- `lifecycle` : Populated by the contract loader at boot. Immutable during runtime.

workspace_class_definition

- `description` : Products define workspace classes in their policy.yaml under workspace_classes. Each class maps standard mount points to scope configurations. The platform reads these definitions at boot and provisions mounts accordingly.

FORMAT

workspace_classes

- `class_name` :
 - `description` : string — human-readable purpose
 - `workspace_scope` : string — per-agent | per-flow-instance

RULES

- `/data` and `/opt/mas/contracts` are always global, read-only. Products cannot override their scope or access.
- `workspace_scope` controls the `/workspace` mount only – the sole configurable dimension.
- Every `workspace_class` referenced by an agent in `agents.yaml` must be defined in `policy.yaml workspace_classes`.
- The platform validates this at boot. Missing class definition is a boot error.

mount_guarantees

- `description` : Rules the runtime MUST enforce.

RULES

- Every agent session executes with all three standard mounts active. No mount may be missing.

- `/data` is the SAME mount point across all workspace classes. An agent in any class sees identical `/data` contents.
- `/workspace` isolation follows the `workspace_scope` declared by the agent's workspace class. per-agent workspaces are never visible to other agents. per-flow-instance workspaces are visible to all agents in that flow instance only.
- Read-only mounts are enforced at the filesystem level (read-only bind mount, or equivalent). Agents cannot write to `/data` or `/opt/mas/contracts`.
- The runtime MUST NOT vary mount availability based on `conversation_mode`, `model_tier`, or any other agent property. Workspace class is the sole determinant of `/workspace` scope.

boot_validation

- `description` : The platform verifies workspace infrastructure at boot.

CHECKS

- Every `workspace_class` referenced in `agents.yaml` is defined in `policy.yaml workspace_classes`
- `/data` mount exists and is readable
- `/opt/mas/contracts` is populated from contract loader
- For each registered agent, `/workspace` path is creatable under the declared scope
- `failure` : Boot aborts if any workspace mount cannot be satisfied. Error message identifies the failing mount, workspace class, and agent.

deployment_mapping

- `description` : How standard mounts map to deployment environments. Mount paths are invariant — only backing storage changes.
- `local_dev` :
 - `/data` : Host directory bind-mounted read-only. Same path for all agent processes.
 - `/workspace` : Host directory under a platform-managed root (e.g., `~/.mas/workspaces/{scope}/{id}`).
 - `/opt/mas/contracts` : Host directory populated by contract loader at boot.

- **docker** :
 - **/data** : Named volume or bind mount, shared across all containers. Read-only flag required.
 - **/workspace** : Per-scope volumes. per-agent classes get per-agent volumes. per-flow-instance classes share an instance volume.
 - **/opt/mas/contracts** : Read-only volume populated by init container or bind mount.
- **production** :
 - **/data** : Network-attached storage (NFS, EFS, GCS FUSE) mounted read-only. Consistent across all compute nodes.
 - **/workspace** : Storage persistence follows `conversation_mode`. task mode: ephemeral (tmpfs acceptable). session modes: persistent storage required.
 - **/opt/mas/contracts** : Read-only volume from deployment artifact.
- **portability_rule** : A product contract that works on local dev MUST work on Docker and production without contract changes. Mount paths are invariant. Only the backing storage implementation changes.